

Missing Physics Discovery through Fully Differentiable Finite Element-Based Machine Learning

Ado Farsi^{*1,2,3}, Nacime Bouziani^{†3,4,5}, and David A. Ham^{‡4}

¹Imperial College London, Earth Science and Engineering Department, London, United Kingdom

²University College London, Earth Sciences Department, London, United Kingdom

³Tanuki Technologies, London, United Kingdom

⁴Imperial College London, Department of Mathematics, London, United Kingdom

⁵Amazon Science UK, London, United Kingdom

ABSTRACT

Modelling physical systems with partial differential equations (PDEs) is central to science and engineering, yet in most real applications the PDE model is incomplete: relationships such as constitutive or thermal laws are unknown. Existing surrogate approaches close this gap by learning the PDE solution from data, sometimes with added physical constraints, but they remain tied to a specific configuration (geometry, boundary conditions, discretisation) and recover the solution rather than the missing physics itself. We introduce FEML, an end-to-end differentiable framework that couples the known PDE (the system’s known physics) with a machine-learned operator for the missing physics. Embedding the PDE solver into training lets this operator be learned directly from the PDE solution, even when its own output cannot be measured—for example, stress when learning constitutive laws. Because the operator, unlike the PDE model, is independent of the system configuration, a law learned in one setting transfers zero-shot to new geometries, boundary conditions, and discretisations, and can be inspected by domain specialists. FEML represents the operator with structure-preserving operator networks (SPONs), which retain key continuous properties at the discrete level and enable learning over complex geometries and meshes. We demonstrate FEML across solid mechanics and thermal transport. From synthetic experiments we progressively discover an elastoplastic law—the nonlinear elastic response, then the plastic hardening law—and compose the two operators into a foundation constitutive model that transfers zero-shot to a three-dimensional torsion problem. Moving to real data, we learn coupled plastic-hardening and ductile-damage laws directly from a benchmark shear-coupon test, reproducing the measured response, including post-peak softening, to within the experimental scatter. Finally, we recover a temperature-dependent conductivity from transient heat-flow data and apply symbolic regression to the learned operator to extract a closed-form law matching the ground truth.

Introduction

Many science and engineering problems rest on well-understood physics yet contain unresolved or missing relationships that are unknown or cannot be readily expressed in mathematical form.

Modelling complex physical systems is traditionally achieved through physics-based models, such as partial differential equations (PDEs). However, such models require exact physical understanding and a fully specified mathematical formulation to accurately describe the physics of interest—conditions that are often difficult to satisfy in practice. Data-driven approaches, particularly machine learning (ML) models, offer an alternative by inferring input–output relationships directly from observable data, enabling prediction of system behaviour even when the governing physics are unknown or incomplete. However, purely data-driven models have their own drawbacks, including the need for large training datasets, predictive power limited to regimes represented in the training data, and reduced interpretability.

Various strategies have been proposed to incorporate physical and mathematical knowledge into machine-learning algorithms^{1–6}. In general, these approaches seek to address the shortcomings of purely data-driven models by training surrogate ML models with both data and physical constraints. Although this reduces the volume of data required, the resulting models typically learn the *solution of a particular PDE configuration* rather than the *unknown operator itself*.

A distinct but related line of research focuses on the direct discovery of physical laws from data. Equation-discovery methods, such as the Sparse Identification of Nonlinear Dynamical Systems (SINDy) framework⁷, use sparse regression over a library of candidate nonlinear functions to recover governing equations in closed symbolic form from time-series

*Email: ado.farsi@imperial.ac.uk

†Email: n.bouziani18@imperial.ac.uk. This work was conducted independently of Amazon.

‡Email: david.ham@imperial.ac.uk

measurements. Recent extensions have incorporated physics-informed priors, encoding known conservation laws or symmetries into neural-network architectures, to discover nonlinear PDE dynamics from scarce or noisy data⁸. In parallel, physics-informed neural networks (PINNs) have been used to calibrate constitutive material models by embedding conservation laws into the loss function⁹. While these approaches share with the present work the objective of uncovering hidden physical relationships, they differ in scope and methodology. Equation-discovery methods require a pre-specified dictionary of candidate terms and are most effective when the governing variables are directly observable; extending them to PDE-governed systems where the operator of interest (e.g., a stress–strain relation) is not directly measurable, and can only be inferred through the PDE solution, is considerably more challenging. PINN-based constitutive calibration, as formulated by Haghighat et al.⁹, operates at the material-point level and requires direct stress–strain data from homogeneous tests as training input; however, stress is an internal quantity that cannot be directly measured in most experimental settings. By contrast, the framework introduced in this work embeds the unknown operator inside a full boundary-value problem solved by the finite element method, and can therefore learn the constitutive law from indirect, experimentally accessible measurements such as displacement fields or boundary forces, without ever requiring direct observation of stress. These lines of work are nonetheless complementary: for instance, having identified a material law via the present framework, one can subsequently apply sparse-identification or symbolic-regression techniques to learn a closed-form expression from the learned neural operator, as we demonstrate in Section 1.3.2.

In many applications, the missing physics does not correspond to an explicit input–output map, but rather manifests as an *internal operator*—a hidden functional relationship (such as a constitutive law or thermal material property) that governs interactions within the PDE system itself. This operator acts locally within the governing equations, coupling quantities (e.g., stresses or heat fluxes) through unknown dependencies that cannot be directly observed or isolated experimentally. Existing methods such as Fourier Neural Operators (FNO) or Physics-Informed Neural Operators (PINO)^{10,11} address a different learning problem: they learn the full *solution map* from PDE inputs to PDE solutions, producing surrogate models for fast evaluation within a given training distribution. In addition, they rely on pointwise discretisations of input–output fields that can discard the underlying continuous function-space structure, leading to inconsistencies and degraded operator approximation¹². While effective as surrogate models, these approaches are designed to approximate the entire configuration-specific mapping rather than to identify the hidden operator itself; consequently, they do not produce a reusable physical law that can be transferred across different problem configurations (e.g., different geometries, boundary conditions, or discretisations) or inspected by domain specialists for scientific interpretation. Similarly, Raissi et al.¹³ showed that physics-informed neural networks (PINNs) can be used for inverse coefficient identification, but their formulation is primarily a pointwise residual minimisation method in strong form, not a variational (weak-form) method. Moreover, it learns the solution and unknown coefficients jointly for a single inverse instance, rather than amortising a reusable operator over a family of configurations; in that sense it does not leverage the function-space and operator-learning structure that later enables discretisation transfer and rapid reuse across configurations. The objective of the present work is instead to recover only the unknown relationship, such that (i) the learned operator can be transferred to other problem configurations, and (ii) the resulting model can be inspected and constrained by domain experts for downstream analysis. Learning such operators from data requires coupling traditional PDE solvers with machine learning models. Since the operator of interest is embedded within the PDE, measurable physical quantities, from which a training loss can be defined, can be obtained by solving the governing equations. This coupling necessitates *end-to-end differentiability* through both the PDE solver and the embedded ML operator, enabling gradients to propagate across the entire system during training².

In this work, we introduce Finite Element-Based Machine Learning (FEML), a general end-to-end differentiable framework for learning missing physics in systems with partially known governing laws. FEML cleanly separates known physics, expressed as PDEs in weak form and discretised with the finite element method (FEM), from unknown relationships, which are represented by ML operators embedded within the FEM solver. By embedding the PDE solver in the training loop and providing end-to-end differentiability, FEML enables learning from indirect measurements when operator inputs/outputs are not directly observable. Our framework employs *structure-preserving operator networks* (SPONs) to model the missing-physics operator. SPONs preserve key continuous properties at the discrete level, enable learning over complex geometries and meshes, and permit zero-shot generalisation across different discretisations (mesh resolutions and/or finite element choices). In addition, SPONs provide theoretical guarantees on operator approximation for a given training discretisation¹⁴.

The FEML framework is also particularly suited when some of the quantities involved in the unknown relationship are not directly measurable, allowing one to learn from data on related, measurable quantities within the same system. An example is the discovery of material constitutive laws. Here, the unknown relationship is that between strains (i.e. material deformation) and stresses (the internal forces reacting to deformation); the latter, although it can be estimated under simplified loading conditions, is not directly measurable. The idea is to use known physical relationships to learn the constitutive law from measurable quantities such as displacements and applied forces (this will be covered in Section 1.2). One may then wish to examine the ML operator on its own to uncover the previously hidden relationships, for example by applying symbolic

regression or sparse-identification techniques (see Section 1.3.2). In the context of constitutive modelling, this could enable the extrapolation of generalised stress–strain laws. One might also leverage the learned operator as a foundation model, trained on data from simplified laboratory tests, to predict material behaviour under different geometries and loading conditions without retraining (see Section 1.2.3). This has potential practical applications in many fields, such as subsurface engineering, where integrating sparse prior knowledge with limited field measurements is critical.

Our framework relies on Firedrake^{15,16} and allows for end-to-end differentiable coupling with ML architectures implemented in PyTorch¹⁷ and JAX¹⁸. To the best of our knowledge, the FEML framework is the first to support the bidirectional coupling of state-of-the-art general finite element solvers and arbitrary machine learning architectures in an end-to-end differentiable manner. In contrast, most existing efforts have focused on deploying the algorithmic differentiation pipelines of machine learning frameworks to yield differentiable physics constraints, often specialised to particular applications (such as *XLB*¹⁹, *PhiFlow*²⁰, *Adept*²¹). A similar approach to ours has been presented in²², although it lacks the adjoint capabilities required for differentiation through constraints based on PDEs, and in⁴, although it is limited to a restricted set of fluid simulations.

This work is organised as follows. Section 1.1 presents our FEML framework for learning missing physics by embedding trainable ML operators within PDE systems. Section 1.2 illustrates its application to solid mechanics through a sequence of experiments that demonstrate how FEML enables the progressive discovery of new physics: a learned physical law can be composed into a new model to discover further, previously inaccessible physical laws. Concretely, the elastic constitutive law learned in a first experiment is frozen and embedded as known physics in a second model, whose sole trainable component is the plastic hardening law that governs material behaviour beyond the elastic limit. The two pretrained operators are then composed into a foundation constitutive model and deployed zero-shot on an unseen three-dimensional problem, illustrating how individually discovered laws can be assembled into increasingly complete physical descriptions. As a demonstration of real-world applicability, the framework is applied to *real* experimental data from a benchmark shear-coupon test, where both a plastic-hardening law and a ductile-damage law are learned directly from the measured response, extending the discovered description to post-peak softening within a finite-strain formulation. Section 1.3 extends the examples to transient thermodynamics by learning nonlinear thermal conductivities from noisy temperature fields, and further demonstrates how a closed-form expression can be learned from the neural operator via symbolic regression (Section 1.3.2). Overall, the examples increase in complexity to progressively build intuition for the proposed framework, from an initial setting with simple geometry and boundary/loading conditions to a final case involving a complex multi-component domain and a time-dependent governing equation with fluctuating boundary conditions. Finally, Section 2 summarises our findings and outlines directions for future work. All examples presented herein can be implemented and executed using the Firedrake finite element framework.

1 Results

1.1 Framework

We introduce the Finite Element-Based Machine Learning (FEML) framework, a general and end-to-end differentiable approach for learning missing physics, that is, unknown relationships between quantities in systems where partial physical knowledge is available. Our main motivation is to learn operators that model such unknown relations. We consider an operator $\mathcal{G} : \mathcal{U} \rightarrow \mathcal{V}$, where $\mathcal{U}$ and $\mathcal{V}$ are (typically infinite-dimensional) Hilbert spaces of functions defined on a bounded domain $\Omega \subset \mathbb{R}^d$ in spatial dimension $d \in \{1, 2, 3\}$. For sake of simplicity, we consider $\mathcal{U}$ and $\mathcal{V}$ to be defined on the same domain Ω , but different bounded domains and spatial dimensions can be considered. We introduce a framework for approximating $\mathcal{G}$ by a learnable operator $\mathcal{G}_\theta : \mathcal{U}_h \rightarrow \mathcal{V}_h$ of parameters θ , where $\mathcal{U}_h$ and $\mathcal{V}_h$ are finite element spaces arising from the discretisation of the spaces $\mathcal{U}$ and $\mathcal{V}$, respectively.

Such operators can traditionally be learned in a supervised manner from direct observations, i.e. a set of input-output pairs $\{(x_i, \mathcal{G}(x_i))\}_i$, with $x_i \in \mathcal{U}$. However, in practice, many input–output signals from unknown operators cannot be accessed directly via experiments, since they form only part of a larger physical model governed by a PDE and consequently, we cannot learn them directly. For example, constitutive laws describing the stress–strain relationship of a material are generally unknown, and the resulting stress tensor, which encodes internal forces under deformation, cannot be measured directly. In contrast, the displacement field is measurable and satisfies a PDE that incorporates this constitutive relation. Therefore, to learn the constitutive law, we embed our ML model within the PDE solver: during training, the ML model predicts the stress tensor required by the solver to compute the corresponding displacement, and we compute a loss against the measured displacement to update the model.

Our aim is to learn $\mathcal{G}_\theta$ under some loss $\mathcal{L}(u_\theta, u^{obs})$, where u_θ is a PDE solution and u^{obs} are observable data, i.e. we have:

$$F(u_\theta, \mathcal{G}_\theta(u_\theta); v) = 0 \quad \forall v \in U_h \quad (1)$$

where (1) is the variational form of the PDE with F the residual. This PDE can be linear or nonlinear, steady or time-dependent. In practice, one can also equip it with boundary conditions but we have simplified the setup description for sake of

simplicity.

We propose a general framework and the loss function $\mathcal{L}$ can be defined in different ways depending on the specific problem. Figure 1 shows a schematic of the proposed framework. For example, in the case of learning a constitutive law in Section 1.2.2, F is the residual of the PDE that describes the equilibrium equation with some boundary conditions in the simulated domain. Moreover, $\mathcal{L}$ quantifies the discrepancy between experimental displacement measurements and the corresponding solution of the FEM solver, and $\mathcal{G}_\theta$ is an unknown constitutive law.

Our framework combines PDE modelling of the physical problem of interest with ML modelling of the operator to approximate. Minimising the loss $\mathcal{L}(u_\theta, u^{obs})$ for training requires computing the gradient of the loss with respect to the parameters, i.e. $\frac{d\mathcal{L}}{d\theta}$, which by chain rule requires the gradient of the PDE solution with respect to the parameters, i.e. $\frac{du_\theta}{d\theta}$, which in turn necessitates the gradient of the ML operator, i.e. $\frac{d\mathcal{G}_\theta}{d\theta}$. In other words, learning $\mathcal{G}_\theta$ requires end-to-end differentiability of the entire system, i.e. being able to differentiate through the PDE solution u_θ and through the ML components of the system to compute $\frac{d\mathcal{L}}{d\theta}$. How this end-to-end gradient flow is realised in practice is detailed in Section 3.1.

Many real-world problems and engineering applications require the use of advanced numerics with state-of-the-art capabilities for PDE modelling. The simulation of complex physical systems by coupling advanced numerics for PDEs with state-of-the-art machine learning demands the composition of specialist PDE solving frameworks with industry standard machine learning tools. Hand-rolling either the PDE solver or the ML model will not cut it. Our framework introduces a generic differentiable programming interface that allows to combine the state-of-the-art Firedrake framework for PDE modelling, with different ML frameworks, including PyTorch¹⁷ and JAX¹⁸ deep learning libraries. As a result, it provides scientists and engineers with an efficient and highly productive way to learn operators combining FEM operations, e.g. solving a PDE using FEM, with ML algorithms, thanks to end-to-end differentiability and benefiting from state-of-the-art performance of both FEM and ML libraries.

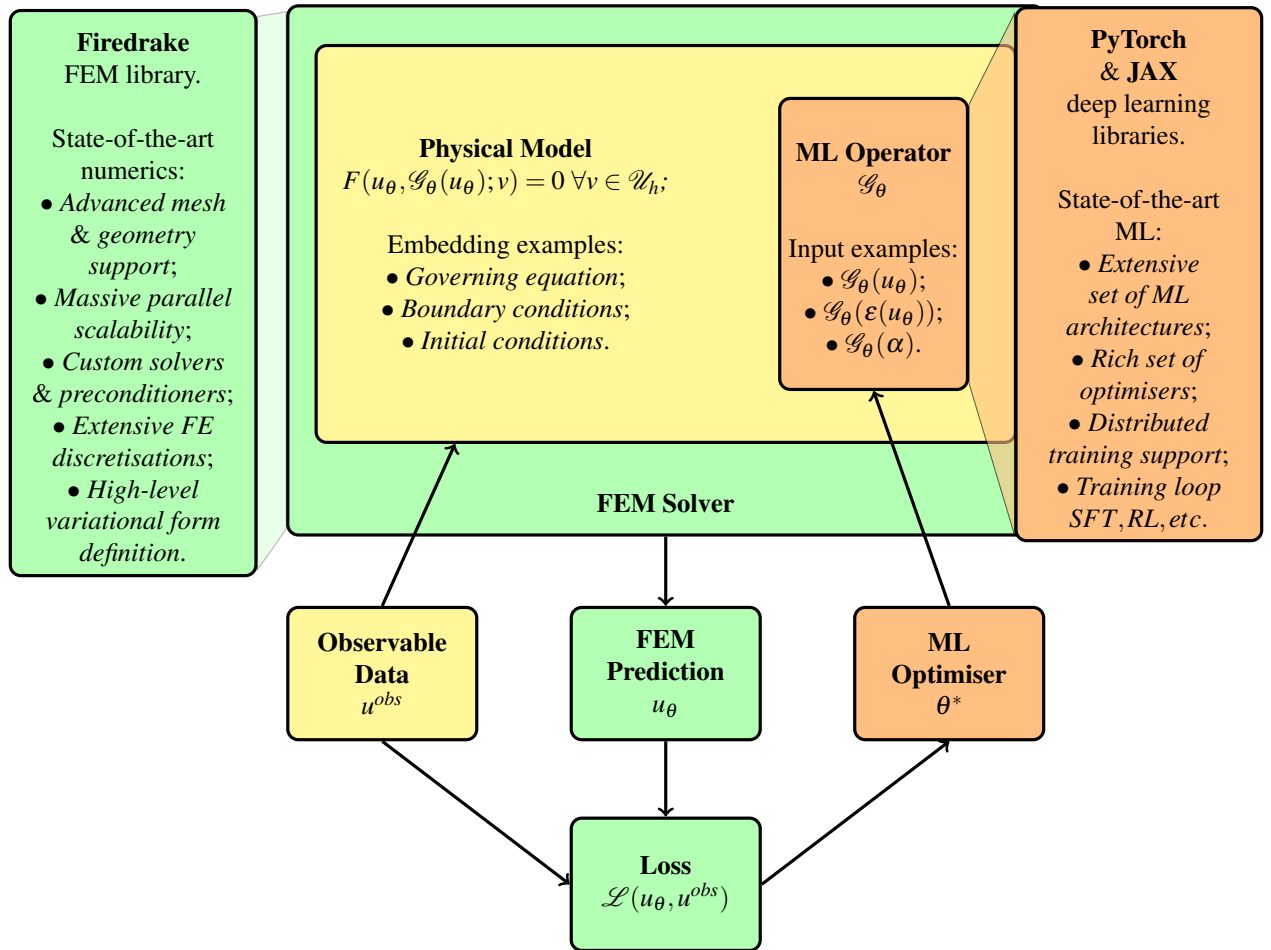


Figure 1. Schematic of the proposed framework for embedding neural networks as trainable operators within PDE systems. The framework combines finite element solvers with machine learning models to learn ML operators from observable data.

1.1.1 Operator Learning over Finite Element Spaces

Our framework differs from the traditional operator learning literature as it embeds the learnable operator $\mathcal{G}_\theta$ into a differentiable FEM solver, thereby allowing end-to-end differentiable FEM-based operator learning. Another notable difference is that our setting entails learning over finite element spaces, i.e. $\mathcal{U}_h$ and $\mathcal{V}_h$ result from finite element discretisations. Consequently, the operator $\mathcal{G}_\theta$ can be expressed as an encode-process-decode architecture¹⁴, with a learnable processor over the spaces induced by the degrees of freedom (DoF) of $\mathcal{U}_h$ and $\mathcal{V}_h$. More precisely, $\mathcal{G}_\theta$ can be defined as

$$\mathcal{G}_\theta(f) = \mathcal{D} \circ \mathcal{P}_\theta \circ \mathcal{E}(f), \quad f \in \mathcal{U}_h, \quad (2)$$

where $\mathcal{E}$ and $\mathcal{D}$ refer to the *encoder* and *decoder*, while $\mathcal{P}_\theta: \mathbb{R}^n \mapsto \mathbb{R}^m$ with $n = \dim(\mathcal{U}_h)$ and $m = \dim(\mathcal{V}_h)$ represents a learnable model of parameters θ , known as the *processor*. The encoder extracts the degrees of freedom of an input function $f \in \mathcal{U}_h$:

$$\mathcal{E}(f) = (f_1, \dots, f_n), \quad (3)$$

where $f_i = \langle f, \phi_i \rangle$ corresponds to the Galerkin projection onto the i -th basis function ϕ_i . On the other hand, the decoder maps the predicted DoFs in $\mathcal{V}_h$ to the reconstructed solution $u \in \mathcal{V}_h$ as

$$\mathcal{D}(u_1, \dots, u_m) = u, \quad (4)$$

with $u(x) = \sum_{i=1}^m u_i \phi_i(x)$, for $x \in \Omega$, and where $(\phi_i)_{1 \leq i \leq m}$ a basis of $\mathcal{V}_h$. Such encode-process-decode operators are referred to as *structure-preserving operator networks* (SPON)¹⁴ as they preserve some key mathematical and physical properties of the operator $\mathcal{G}$ at the discrete level and offer explicit trade-off between accuracy and efficiency. Our framework inherits the properties of SPON such as zero-shot super-resolution and theoretical bounds on the approximation error of $\mathcal{G}_\theta$.

Zero-shot super resolution. $\mathcal{G}_\theta$ outputs a finite element function $u \in \mathcal{V}_h$ that can be evaluated at any point x in the geometrical domain Ω , i.e. by simply using $u(x) = \sum_{i=1}^m u_i \phi_i(x)$. This property results from the FE discretisation of $\mathcal{U}$ and $\mathcal{V}$ and holds on complex geometries and independently of the mesh and resolution $\mathcal{G}_\theta$ was trained on. This relation is crucial for transferring solutions between different meshes and spatial discretisations, such as for zero-shot super-resolution, and it enables architectures that can seamlessly operate across multiple resolutions. In practice, implementing such a property for complex geometries and/or non-trivial FE discretisations is challenging. However, our differentiable programming coupling of Firedrake and ML software allows to implement that in a single line of code for arbitrary meshes and a wide range of FE discretisations.

Approximation error. Let Ω be an open bounded domain of $\mathbb{R}^n$, $\mathcal{V} = H^k(\Omega)$ and $\mathcal{U} \subset H^k(\Omega)$. Let $\mathcal{G}: H^s(\Omega) \rightarrow \mathcal{V}$ be a Lipschitz continuous operator for some $0 \leq s \leq k$ and $0 < \varepsilon < 1$, and $\mathcal{U}_h$ and $\mathcal{V}_h$ be conforming finite element spaces. Under mild assumption on $\mathcal{U}$ and assuming $\mathcal{U}_h$ and $\mathcal{V}_h$ satisfy the standard finite element hypotheses²³, one can show that there exists a learnable operator $\mathcal{G}_\theta: \mathcal{U}_h \rightarrow \mathcal{V}_h \subset \mathcal{V}$ with a number of parameters bounded

$$|\theta| < C_1 \varepsilon^{-C_2/h^{k^n}} (\log(1/\varepsilon) + 1),$$

such that for all $f \in \mathcal{U}$,

$$\|(\mathcal{G} - \mathcal{G}_\theta \circ P_{\mathcal{U}})(f)\|_{H^s(\Omega)} \leq C_3 h^{k-s} \left(\|f\|_{H^k(\Omega)} + \|\mathcal{G}(f)\|_{H^k(\Omega)} \right) + \varepsilon(h), \quad (5)$$

where $P_{\mathcal{U}}: \mathcal{U} \rightarrow \mathcal{U}_h$ is the Galerkin interpolation. We refer to¹⁴ for the general approximation theorem and its proof. The first term denotes the finite element error on the input-output spaces, which can be controlled via the finite element discretisation, e.g. a higher polynomial degree k results in a higher convergence rate via h^{k-s} . On the other hand, the second term reflects the neural network's approximation error, which can be reduced by increasing the number of parameters. The left-hand side in eq. (5) can be seen as an operator aliasing error¹², which can be explicitly controlled by the mesh resolution and the discretisation of the input-output spaces. It is worth noting that higher-order discretisations comes with higher convergence rate but also with an additional computational cost as it increases the number of DoFs and therefore the size of the input-output of the learnable processor $\mathcal{G}_\theta$. This trade-off between accuracy and efficiency is well known in the FEM literature and is inherited by our framework.

In the following sections, we present examples of the proposed framework applied to solid mechanics and thermodynamics. Section 1.2 focuses on learning nonlinear constitutive laws, while Section 1.3 demonstrates the learning of nonlinear thermal properties in transient heat conduction problems.

1.2 Learning Materials Constitutive Laws

In this section, we demonstrate the application of the proposed framework to solid mechanics and materials science. Different materials exhibit different deformation behaviours under applied forces. For instance, the relationship between deformation and the internal forces can vary significantly when studying rocks, metals, or other engineered metamaterials. Accurately characterising these relationships is crucial in many engineering domains, from the design of resilient infrastructure in civil engineering to the development of aircraft structures in aeronautical engineering.

We consider an unknown material, representative of a closed-cell polymeric foam, whose full elastoplastic response must be discovered from experimental data. The material exhibits two forms of nonlinearity: (i) an elastic softening modulus that degrades under compressive volumetric strain, and (ii) plastic hardening governed by J2 flow with a Voce-type yield law. Rather than attempting to learn both nonlinearities at once, the experiments are designed to follow a *progressive discovery* strategy in which each stage isolates and learns a single piece of unknown physics, building on the knowledge acquired in the previous stage. In Section 1.2.1, the material is loaded within its elastic regime via a displacement-controlled uniaxial test, so that the only unknown is the nonlinear elastic response; this first experiment learns the Young's modulus $E(\kappa)$ from measured reaction forces. In Section 1.2.2, the learned elastic operator is frozen—thereby removing the elastic response from the set of unknowns—and the material is loaded beyond its elastic limit in a Brazilian disc test, so that the only remaining unknown is the plastic hardening law; this second experiment also illustrates how *a priori physical knowledge* can be encoded directly into the ML architecture: here, we assume that the material hardens monotonically under continued plastic deformation—an assumption that, in a real-world setting, might stem from preliminary test data, domain expertise, or theoretical considerations. A monotone neural network is used to parameterise the yield-stress evolution $\sigma_y(p)$, guaranteeing this property by construction rather than relying on a soft penalty or post-hoc projection. The learned yield-stress evolution is obtained from full-field displacement measurements. In Section 1.2.3, the two pretrained operators are combined into a foundation constitutive model and deployed zero-shot on a three-dimensional cylindrical rod with a transverse keyhole-shaped through-slot subjected to torsion—a problem that differs from the training setup in geometry, dimensionality, and loading.

The four preceding experiments use synthetic data, which provides an exact ground-truth operator against which the learned model can be quantitatively verified. As a demonstration of real-world applicability, Section 1.2.4 applies the framework to *real* experimental data from a benchmark shear-coupon test on a titanium alloy, where the constitutive law is genuinely unknown. This example serves two roles: it extends the discovered description beyond elasticity and hardening to *ductile damage*—a second learned operator that captures post-peak softening within a finite-strain formulation—and it demonstrates that the framework operates on real laboratory measurements, not only synthetic ones. The two settings are complementary: the synthetic studies verify the method against known ground truth, while the real-data study demonstrates its practical applicability where no ground truth exists.

The formulation of the solid mechanics problem with the embedded ML constitutive model is detailed in Section 3.2.

1.2.1 Learning Nonlinear Elastic Laws from Loads in Displacement-Controlled Experiments

This first experiment targets the elastic regime of the material, where the sole unknown is the nonlinear relationship between strain and stress in the absence of permanent deformation. By restricting the loading to remain below the yield stress, plastic effects are excluded and the learning problem reduces to discovering the elastic constitutive law alone. The elastic operator learned here will serve as a fixed, known component in the subsequent plasticity experiment (Section 1.2.2).

We focus on displacement-controlled uniaxial tests—standard experiments in materials science, rock mechanics, and civil engineering to assess the compressibility and strength of materials.

We idealise the sample as a 2D rectangular domain, as depicted in Figure 2a. The two vertical sides of the sample are free, while the bottom and top surfaces have imposed displacements. The bottom surface is fixed throughout the simulation. A prescribed displacement $\bar{u}_i$ is applied to the top surface, where i is an integer from 1 to N that represents the sequence of displacements applied during the experiment. The loading is kept within the elastic regime of the material (i.e. the maximum stress remains below the yield stress σ_{y0}). This configuration simulates the quasi-static loading condition in which time increments correspond to sequential deformation steps rather than physical time. Figure 3 illustrates the mathematical definition of the problem and the proposed framework for learning constitutive laws from displacement-controlled experiments. The loss function is computed as:

$$\mathcal{L} = \frac{1}{N} \sum_i^N \frac{|F_i^{\text{fem}} - F_i^{\text{obs}}|}{|F_i^{\text{obs}}|} \quad (6)$$

where F_i^{fem} is the applied load predicted by the FEM model with the ML-based constitutive model at time step i , and F_i^{obs} is the corresponding synthetic experimental load. It is important to note that the Poisson effects are not taken into account in this definition of the loss function and therefore the constitutive law is not univocally defined. In this example, the Poisson's ratio is assumed known ($\nu = 0.3$). This simplification is adopted because the uniaxial test provides no lateral displacement information

from which v could be inferred. In principle, v could be learned jointly with E by augmenting the loss with horizontal displacement measurements—for example from a digital image correlation (DIC) system—but we omit this additional data channel here to keep the example focused on the elastic softening law. A more general architecture that can learn both Lamé parameters from full-field displacements is used in Section 1.2.2. More details about the ML constitutive model architecture, the symmetries enforced by the architecture, and the physical constraints it does and does not guarantee are described in Section 3.2.2.

The synthetic experimental data is generated by solving the same PDE system with a known constitutive law: a nonlinear elastic softening law representative of a closed-cell polymeric foam (see Section 3.2.1 for the explicit form). The training dataset consists of six force-displacement pairs, with four additional pairs used for model validation. To represent real data, a 1% noise is added to the scalar force values in the synthetic experimental data.

Figure 4 shows the training and validation loss curves together with the evolution of the force-displacement response predicted by the FEM solver incorporating the ML-based constitutive model at successive training stages (epochs 0, 50, and 200). The dotted line denotes the reference force-displacement curve, while the blue line indicates the model prediction. At initialisation (epoch 0) and after 50 epochs, the ML-based model captures an almost linear elastic response. After 200 epochs, it accurately reproduces the material's softening behaviour, demonstrating its ability to learn general nonlinear constitutive responses from a minimal training dataset.

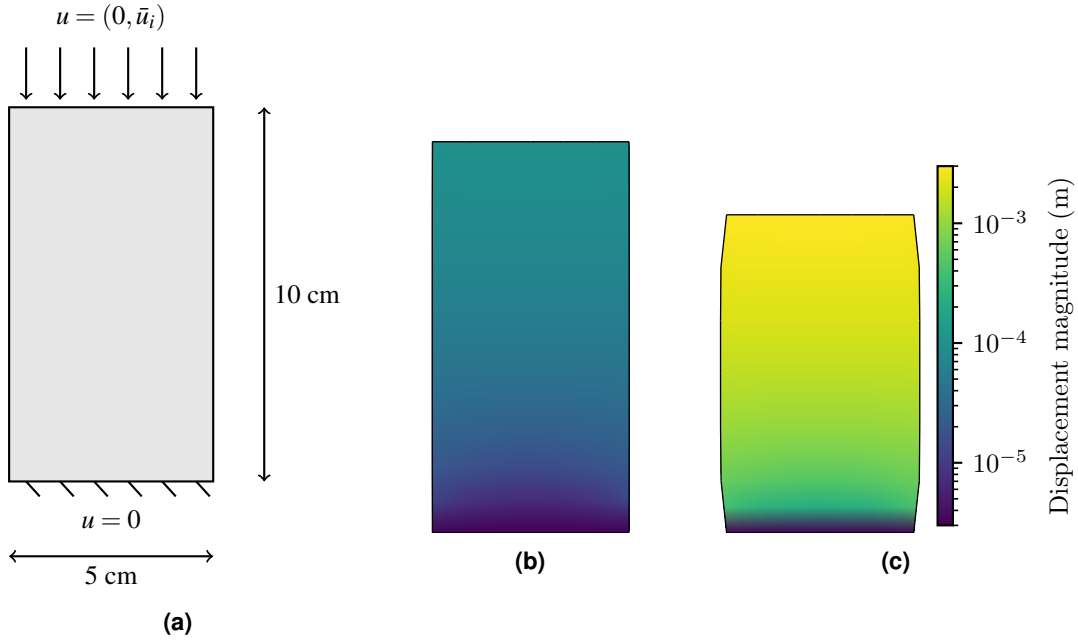


Figure 2. Displacement-controlled uniaxial test: (a) schematic of the 2D test setup with free vertical boundaries, a fixed bottom, and a prescribed displacement $\bar{u}_i$ at the top; displacement magnitude at the (b) first and (c) last loading increment of the training data. The deformation has been amplified by five times for visualisation.

1.2.2 Learning Plastic Hardening Laws from Displacements in Load-Controlled Experiments

Having established the elastic constitutive law in the previous experiment, we now proceed to the second stage of the progressive discovery strategy: learning what happens when the material is loaded beyond its elastic limit and begins to deform plastically. The elastic operator $E(\kappa)$ learned in Section 1.2.1 is frozen—its parameters are no longer updated—so that the only remaining unknown is the plastic hardening law. This separation is the key advantage of the progressive approach: by building on previously acquired knowledge, each new experiment can focus on a single unknown, reducing ambiguity and improving the identifiability of the learned operator.

Beyond progressive discovery, this experiment also demonstrates how a priori knowledge of the expected physical behaviour can be embedded directly in the ML architecture. In many practical situations, one may have qualitative prior knowledge about the unknown physical law—for example, from preliminary experiments, theoretical arguments, or domain expertise. Here, we assume that the yield stress increases monotonically with accumulated plastic strain (i.e. the material exhibits strict isotropic hardening); in other applications, analogous priors might take different forms, such as convexity of a free-energy potential or positivity of a transport coefficient. Rather than treating this as an unconstrained regression problem and hoping the

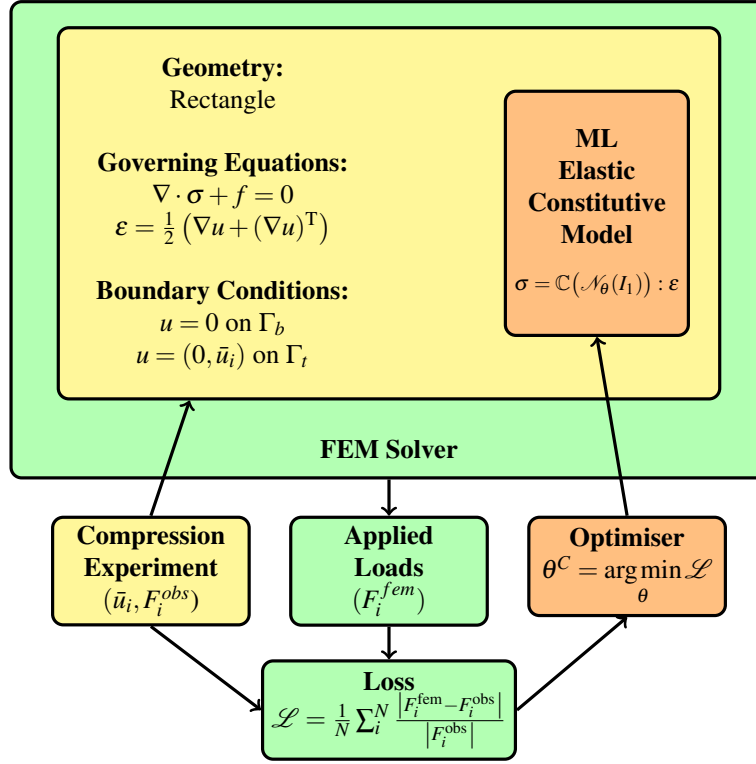


Figure 3. Schematic of the mathematical definition of the problem and the proposed framework for learning constitutive laws from displacement-controlled experiments.

network discovers monotonicity from data, we encode this prior by parameterising $\sigma_y(p)$ with a monotone neural network whose architecture *guarantees* non-decreasing output by construction. This design choice reduces the hypothesis space to physically admissible hardening laws, improves data efficiency, and ensures thermodynamic consistency of the learned plastic response.

We consider the *Brazilian disc test*, a standard method widely used in engineering to indirectly measure the tensile strength of materials. During the test, the displacement field is typically recorded using digital image correlation²⁴. The experiment is performed under force control, where the applied load $\bar{F}_i$ is prescribed rather than the displacement. The same unknown foam is now loaded beyond its elastic limit so that the disc undergoes J2 elastoplastic deformation. Because J2 plastic flow is isochoric ($\text{tr}(\varepsilon^p) = 0$, where ε^p is the plastic strain; the elastoplastic kinematics are defined in Section 3.2.2), the frozen elastic operator $E(\kappa)$ continues to receive the *total* volumetric strain $\kappa = \langle -\text{tr}(\varepsilon) \rangle$ as input, avoiding a circular dependence on the elastic–plastic strain decomposition.

In this example, the sample is idealised as a 2D circular disc, as shown in Figure 5a. The bottom of the disc is fixed throughout the simulation, while a force $\bar{F}_i$ is applied at the top of the sample, where i is an integer from 1 to N that represents the sequence of forces applied during the test. Similarly to the previous example, this configuration simulates the quasi-static loading condition in which time increments correspond to sequential loading steps rather than physical time.

Figure 6 illustrates the mathematical definition of the problem and the proposed framework for learning constitutive laws from load-controlled experiments. The only trainable operator is the yield-stress function $\sigma_y(p)$; all other components—including the frozen elastic modulus $E(\kappa)$ —are fixed during training. The parameters of the neural network-based constitutive model are learned by minimising the following loss function:

$$\mathcal{L} = \frac{1}{N} \sum_i^N \frac{|u_i^{\text{fem}} - u_i^{\text{obs}}|}{|u_i^{\text{obs}}|}, \quad (7)$$

where u_i^{fem} represents the displacement field predicted by the FEM solver with the ML-based constitutive model at loading step i , and u_i^{obs} is the corresponding synthetic experimental displacement field.

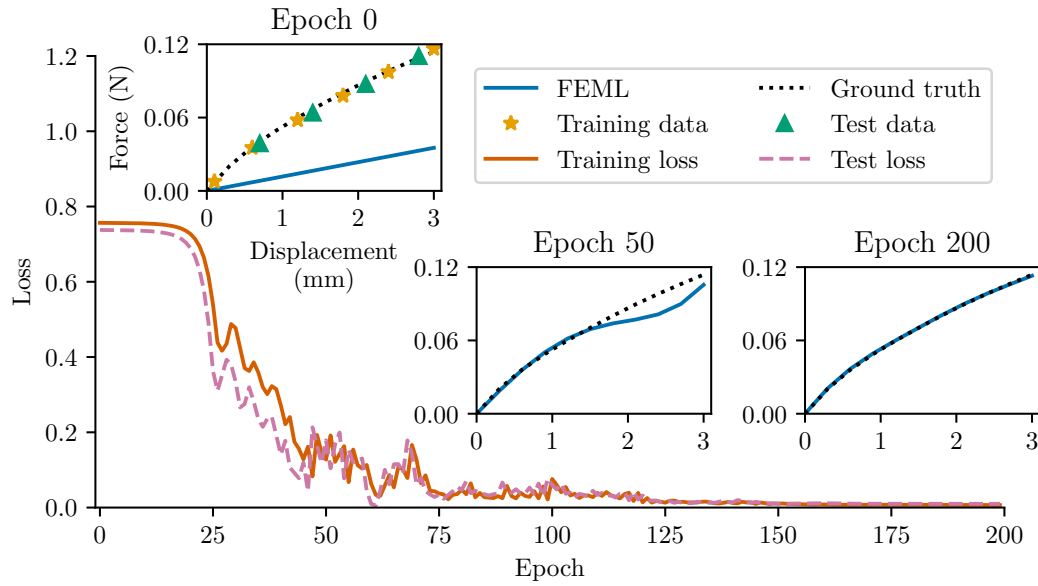


Figure 4. Training loss curve and force-displacement response at different training stages. The figure presents the training and test loss curve alongside the force-displacement response predicted by the FEM solver incorporating the ML-based constitutive model at training epochs 0, 50, and 200. In the plot corresponding to epoch 0, star markers denote the six force-displacement data points used for training, and triangle markers denote the 4 values used for model validation (test). The dotted line represents the reference (ground truth) force-displacement curve, while the blue line corresponds to the response predicted by the FEM solver employing the learned constitutive model. Owing to the universal approximation properties of MLs, the model initially approximates the material behaviour as an almost linear elastic response (epoch 0 and 50). After 200 epochs, the ML-based constitutive model accurately captures the softening behaviour of the material, achieving a close match to the reference curve despite being trained on only six data points.

The synthetic experimental data is generated by solving the same PDE system with the known foam constitutive law: the elastic softening law of Section 1.2.1 combined with a Voce isotropic hardening law for the yield stress (see Section 3.2.2 for the explicit forms). Because full-field displacement data are available, the Poisson's ratio is in principle identifiable; however, we retain $\nu = 0.3$ as a known constant for consistency with the elastic-regime experiment. The training dataset consists of 15 force-displacement field pairs, with 8 additional pairs used for model validation. To represent real data, a 1% noise is added to each nodal value of the displacement fields in the synthetic experimental data, providing a substantially richer perturbation than the scalar noise of the previous example. The architecture of the monotone neural network used to parameterise $\sigma_y(p)$ is described in Section 3.2.2.

Figure 7 shows the training and test loss curves together with the evolution of the load versus maximum displacement magnitude response predicted by the FEM solver incorporating the ML-based constitutive model at successive training stages (epochs 0, 50, and 400). The dotted line denotes the reference load-displacement curve, while the blue line indicates the model prediction. Both the training loss and test error decrease steadily over 400 epochs, indicating convergence of the learned hardening law. Initially, the untrained model (epoch 0) overestimates deformations under applied loads, progressively improving its accuracy. By epoch 400, the ML-based constitutive model closely matches the reference response, benefiting from the richer training data comprising nodal displacement fields at 15 load increments. Compared to the previous example, each node undergoes a distinct strain-stress state, enriching the training dataset and enhancing the constraints on the ML-based constitutive model.

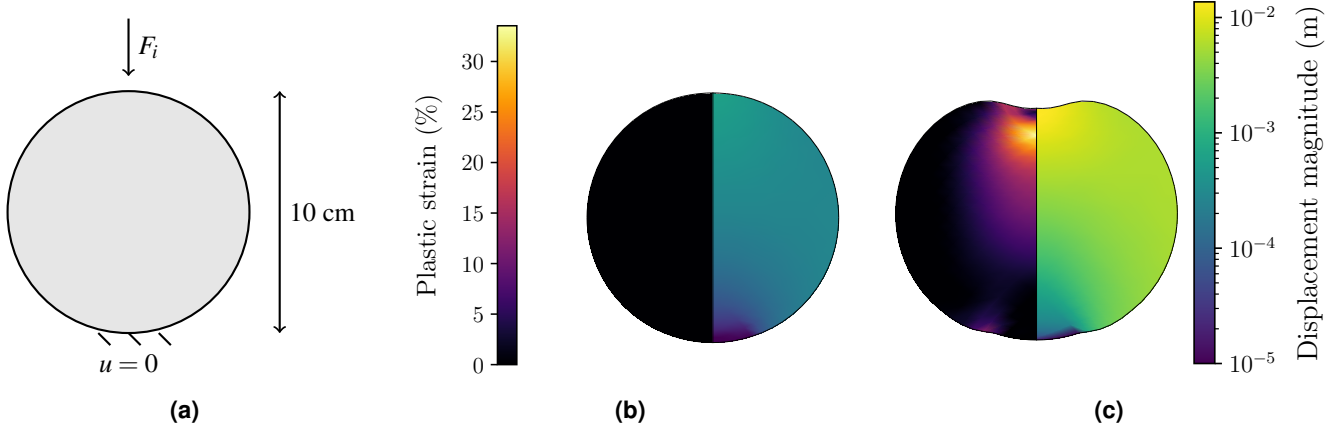


Figure 5. Load-controlled Brazilian disc test: (a) schematic of the test setup with a fixed bottom and a prescribed force F_i at the top; (b) first and (c) last loading increment of the training data. The left half of each disc shows the accumulated plastic strain p and the right half shows the displacement magnitude.

1.2.3 Zero-Shot Inference with an Elastoplastic Foundation Model

We demonstrate the portability of our pretrained constitutive model by deploying it zero-shot on a problem that differs from the training setup. We treat the constitutive operator as a part of a foundation model: a reusable, data-driven component pretrained once and then inserted into new simulations without retraining or architectural changes, while allowing modifications to the surrounding governing PDEs, geometry, and/or boundary/initial conditions.

The foundation constitutive model combines the two operators learned from the material: (i) the elastic modulus $E(\kappa)$ from Section 1.2.1 and (ii) the hardening law $\sigma_y(p)$ from Section 1.2.2. Both are frozen and embedded inside a three-dimensional J2 elastoplastic solver for a cylindrical rod with a transverse keyhole-shaped through-slot at its mid-span, subjected to torsion. The smooth circular cross-section of the rod has zero Saint-Venant warping function, so prescribing rigid rotations on the end faces introduces no warping stresses, and plastic flow is confined to the neighbourhood of the slot rather than the boundary surfaces. As shown in Figure 8a, the two end faces are rotated in opposite directions by $\pm\theta/2$ with $\theta = 3.5^\circ$, yielding a relative twist of θ between the ends and producing a strongly heterogeneous stress field around the slot. The applied twist is deliberately calibrated so that the maximum accumulated plastic strain remains within the range covered by the training data of Section 1.2.2 (peak $p \approx 4\%$), thereby exercising the learned constitutive operator strictly inside its training envelope. Plastic flow develops in approximately 5.9% of the domain. Figures 8b and 8c show, respectively, the displacement magnitude and the von Mises stress on the deformed configuration obtained with the ground-truth constitutive model; the latter also includes an enlargement of the slot region coloured by the accumulated plastic strain p , where plastic flow concentrates.

The problem formulation is summarized in Figure 9. Standard FEM provides the kinematics, balance laws, and boundary/loading conditions. The two pretrained operators supply the elastic and plastic responses from the computed strain and

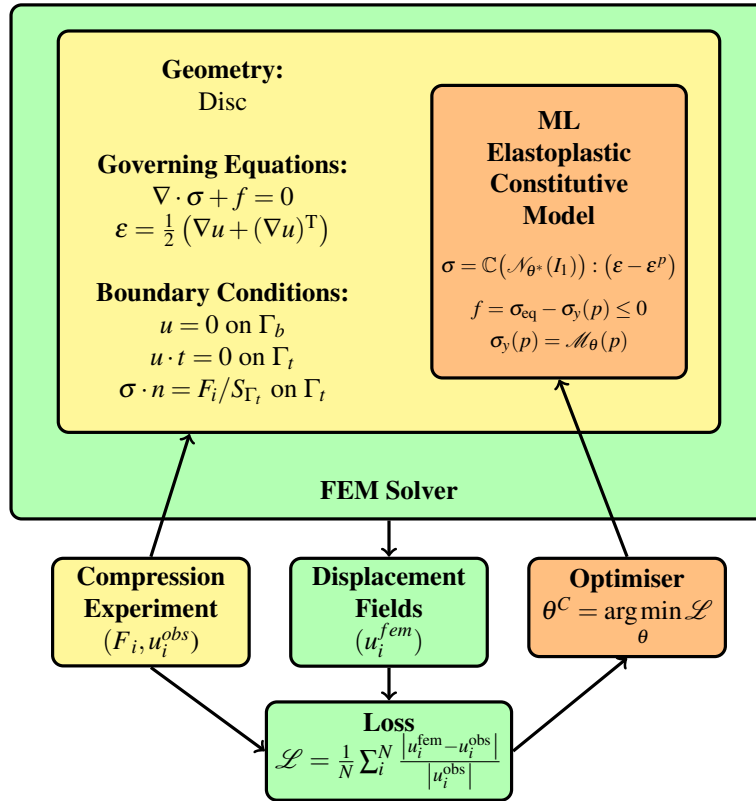


Figure 6. Schematic of the mathematical definition of the problem and the proposed framework for learning constitutive laws from load-controlled experiments.

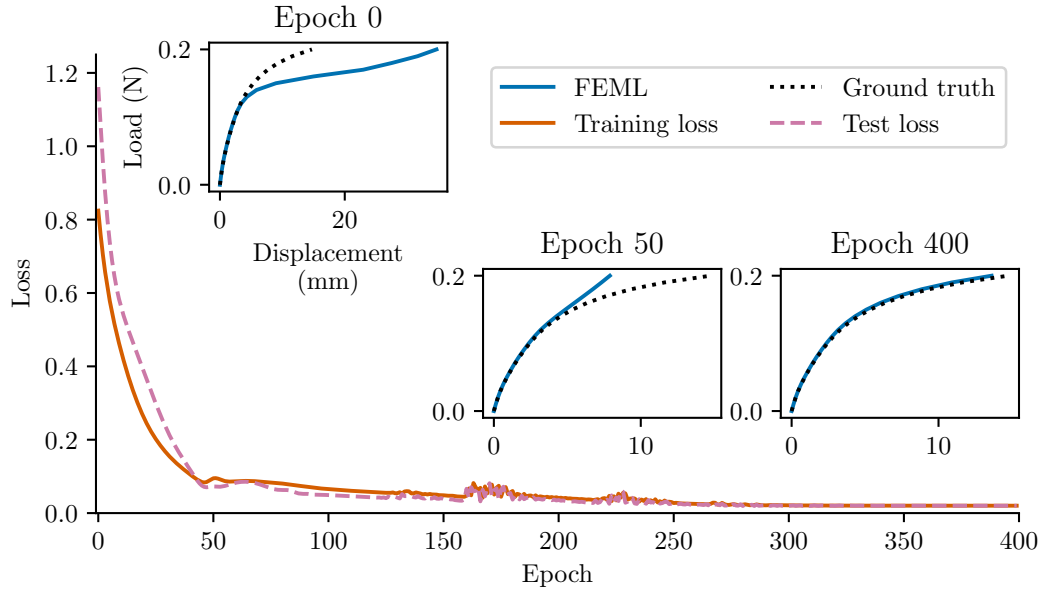


Figure 7. Training loss curve and load versus maximum displacement magnitude response at different training stages. The figure presents the training and test loss curves alongside the load–displacement response predicted by the FEM solver incorporating the ML-based constitutive model at training epochs 0, 50, and 400. The dotted line represents the reference (ground truth) load–displacement curve, while the blue line corresponds to the response predicted by the FEM solver employing the learned constitutive model. Initially, the untrained model (epoch 0) overestimates deformations under applied loads. As training progresses, the model improves its accuracy; after 400 epochs, the ML-based constitutive model closely matches the reference response.

history fields, thereby closing the system without additional calibration.

Figure 10 reports the absolute error fields at the final simulation step between the foundation-model prediction and the synthetic ground-truth solution, viewed from the front and from the side, with black mesh edges overlaid consistently across all three panels. The maximum absolute errors are 2.30×10^{-2} mm for displacement, 0.23 MPa for von Mises stress, and 0.5% for accumulated plastic strain. These margins are particularly compelling given that inference is performed in a different loading regime on a geometry with strong stress-localising features (the keyhole slot), while the constitutive operators were trained on noise-corrupted data. Together, these results indicate that the learned operators transfer robustly to a new geometry, three-dimensional setting, and loading regime—including plastic deformation—in a zero-shot manner. This portability highlights the utility of embedding pretrained operators within PDE solvers to enable efficient simulation across varied engineering scenarios.

1.2.4 Learning Plasticity and Damage from Real Shear-Coupon Experiments

The preceding experiments established and validated the framework on *synthetic* data, where an exact ground-truth operator is available for quantitative verification. We now apply FEMML to *real* experimental measurements, where the underlying constitutive law is genuinely unknown. Beyond demonstrating real-world applicability, this example is the most demanding in the constitutive-modelling sequence in two respects. First, it extends the discovered material description beyond elasticity and plastic hardening to *ductile damage*—the progressive loss of load-carrying capacity that produces post-peak softening and eventual failure—which is represented here by a *second* learned operator embedded in the solver alongside the hardening law. Second, it does so within a *finite-strain* formulation, departing from the small-strain setting of the previous solid-mechanics examples. Because no ground-truth law exists for real data, success is assessed by whether the model reproduces the measured response to within the experimental specimen-to-specimen scatter.

We use the shear-coupon experiment from the Second Sandia Fracture Challenge (SFC2)²⁵, a community benchmark designed to test the predictive limits of computational fracture mechanics for a titanium alloy (Ti–6Al–4V). The specimen is a notched coupon loaded in shear by two grips: the left grip is held fixed while the right grip is displaced parallel to the notch ligament by a prescribed amount $\bar{u}_i$, where i indexes the sequence of imposed grip displacements. The two opposing notches localise the deformation into the ligament between them, producing a near-pure-shear stress state and driving the specimen through yielding, plastic flow, and eventual softening as damage accumulates (Figure 11a). The training target is the measured shear load–displacement response of four nominally identical specimens (VA1, VA2, VP2, VP6); we target the specimen-mean

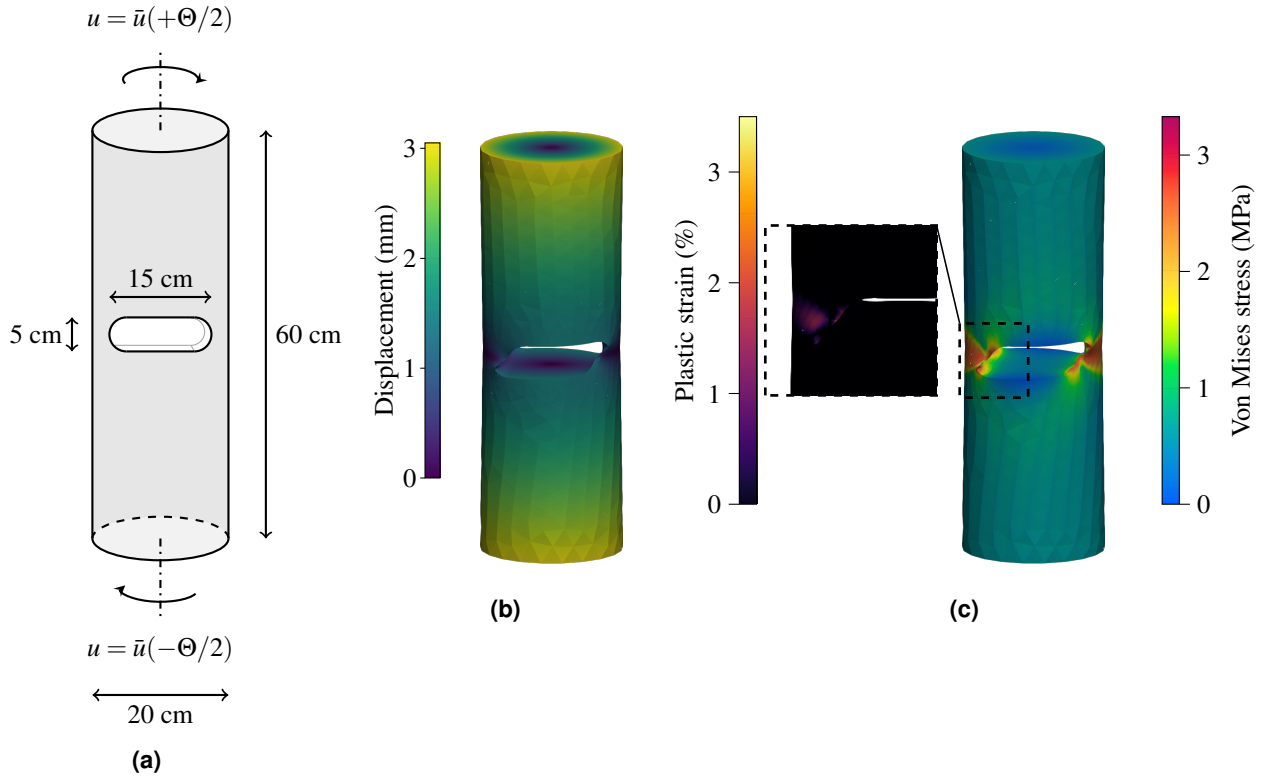


Figure 8. Cylindrical rod with a transverse keyhole-shaped through-slot subjected to torsion: (a) schematic of the rod geometry, viewed from the front so that the stadium-shaped through-slot is visible at mid-span. Rigid rotations of equal magnitude and opposite sign, $\pm\Theta/2$, are prescribed on the two circular end faces, producing a relative twist of $\theta = 3.5^\circ$ between the ends. Deformed configuration from the ground-truth constitutive model, coloured by (b) the displacement magnitude, and (c) the von Mises stress on the full cylinder together with an enlargement (left) of the slot region—highlighted by the red rectangle—coloured by the accumulated plastic strain p , where plastic flow concentrates. The deformation has been amplified by a factor of ten for visualisation.

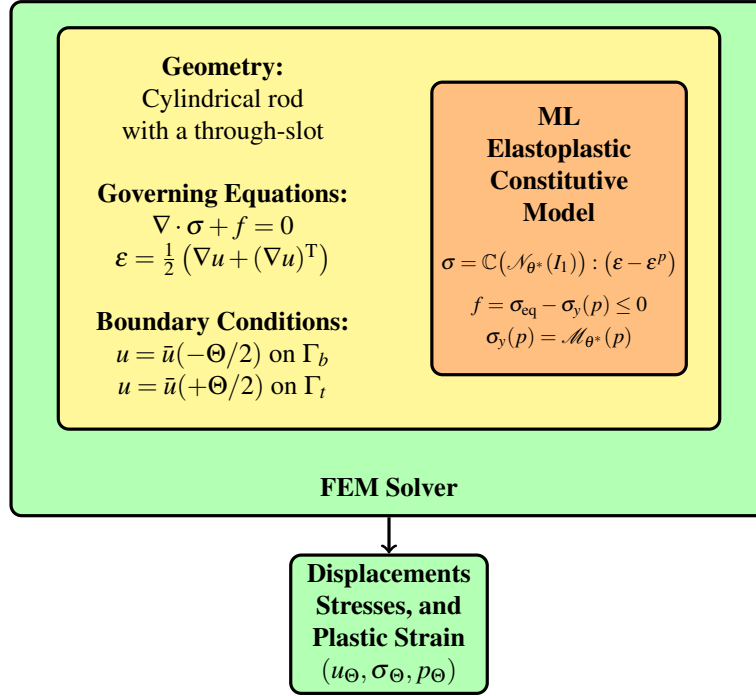


Figure 9. Schematic of the foundation model with the pretrained constitutive operator for zero-shot transfer to a three-dimensional cylindrical rod with a transverse through-slot under torsional loading.

force and weight the loss by the specimen-to-specimen scatter. The mean response reaches a peak of approximately 28.7 kN before softening (Figure 11b).

Both unknown material functions—the plastic hardening law $\sigma_y(p)$ and the ductile-damage law—are represented by learnable neural operators embedded in the finite element solver and trained end-to-end through the adjoint, reusing the progressive-discovery strategy of the previous examples. In the first stage, the elastic constants are fixed (the Young’s modulus $E = 115$ GPa is taken from an independent tensile gauge) and the hardening law $\sigma_y(p)$ is learned on the rising branch of the curve, with damage disabled; as in Section 1.2.2, $\sigma_y(p)$ is parameterised by a monotone neural network that guarantees a non-decreasing yield stress by construction. In the second stage, the hardening operator is frozen and the only remaining unknown is the damage law, represented by a *rectified integral network* (Section 3.2.4) whose architecture enforces the qualitative properties expected of a ductile-damage hazard—a dormant plateau below a strain onset, then monotone, accelerating growth—by construction. The damage is regularised by a nonlocal (implicit-gradient) length scale to prevent spurious mesh-dependent localisation, and is learned on the full curve, including the post-peak softening branch. Because a shear coupon samples essentially a single stress state along each material point’s path, the triaxiality dependence of a general damage model is not identifiable from this experiment; the reduced scalar form adopted here, driven by the accumulated plastic strain p , is therefore both sufficient and well-posed. The finite-strain formulation and both network architectures are detailed in Section 3.2.4.

Figure 12 summarises the problem and the framework. Both stages minimise the same scatter-normalised mean-squared residual between the simulated and measured shear force,

$$\mathcal{L} = \frac{1}{N} \sum_i^N \left(\frac{F_i^{\text{fem}} - F_i^{\text{obs}}}{\sigma_{\text{eff},i}} \right)^2, \quad (8)$$

where F_i^{fem} is the reaction force predicted by the FEM solver at grip displacement $\bar{u}_i$, F_i^{obs} is the experimental mean, and $\sigma_{\text{eff},i} = \sqrt{\text{std}_i^2 + (0.1 F_{\text{peak}})^2}$ normalises each residual by the measured specimen-to-specimen scatter—the standard deviation std_i across specimens at step i , with a floor of 10% of the peak force so that low-scatter points near zero do not dominate the fit. A residual of unity thus corresponds to a prediction within one experimental standard deviation.

Figure 13 reports both training stages. Figure 13a shows the hardening stage: the loss decreases steadily and the predicted load–displacement curve converges to the measured rising branch, with the boxed insets at the initial and final epochs showing the elastic knee and hardening shoulder being captured. Figure 13b shows the damage stage on the full curve: at initialisation

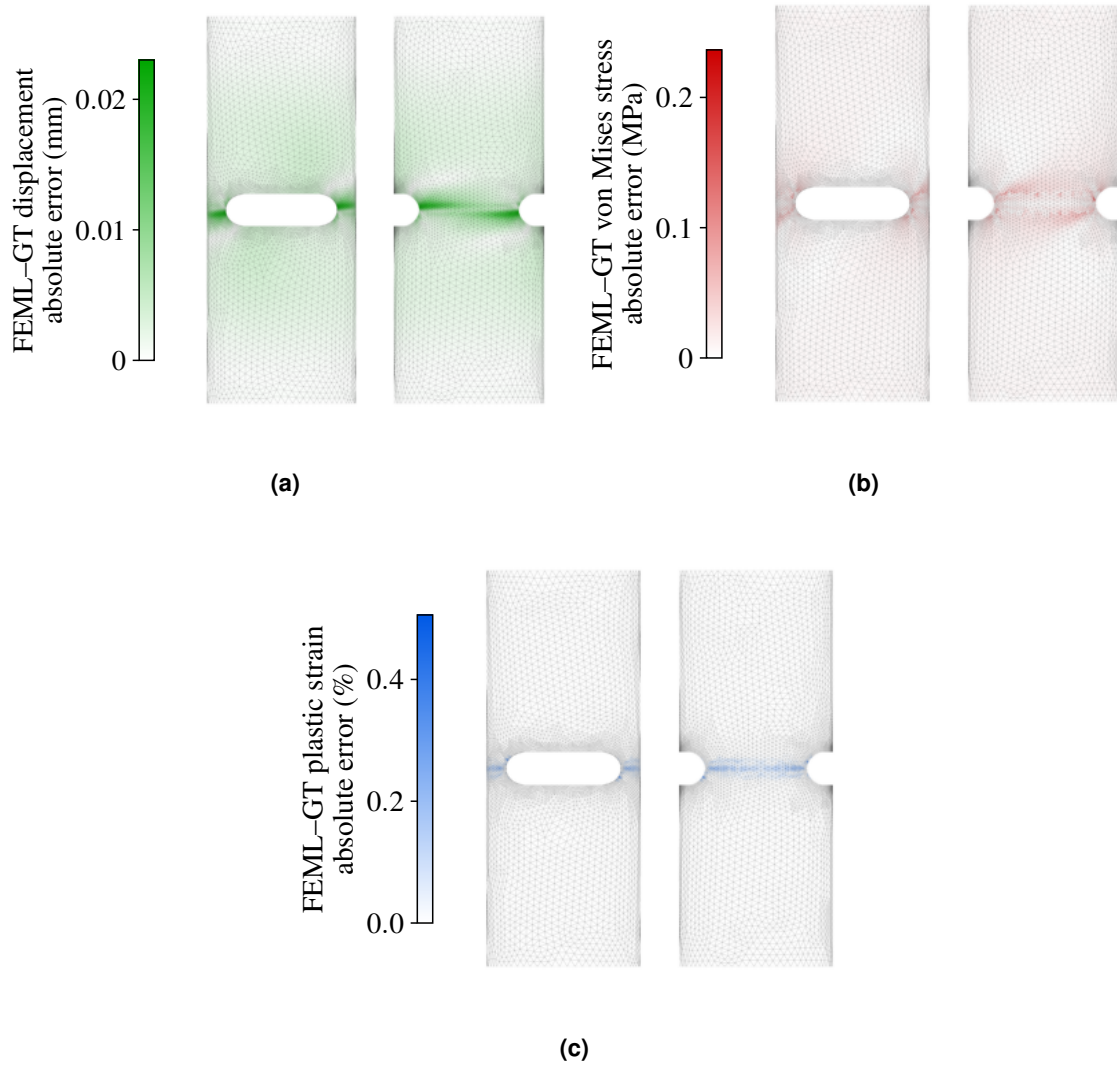


Figure 10. Absolute error fields at the end of the simulation, shown with two views (front and side): (a) absolute displacement difference, (b) absolute von Mises stress difference, and (c) absolute accumulated plastic-strain difference in %.

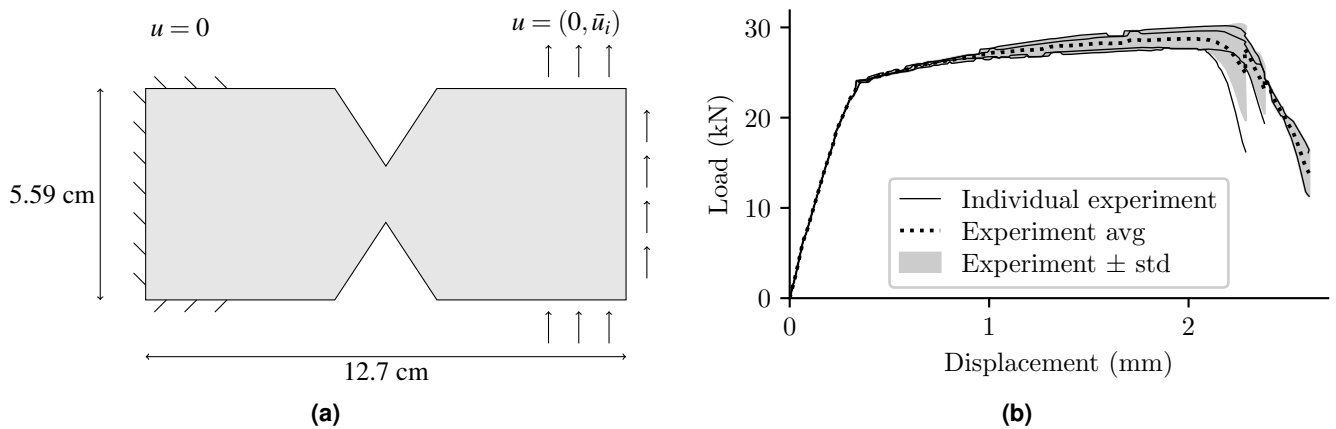


Figure 11. Shear-coupon experiment (SFC2): (a) notched-coupon geometry and boundary conditions—the left grip is fixed ($u = 0$) and the right grip is displaced by $u = (0, \bar{u}_i)$, localising shear in the ligament between the two notches; (b) measured shear load-displacement curves for the four specimens, with the specimen mean and the $\pm$ one-standard-deviation scatter band.

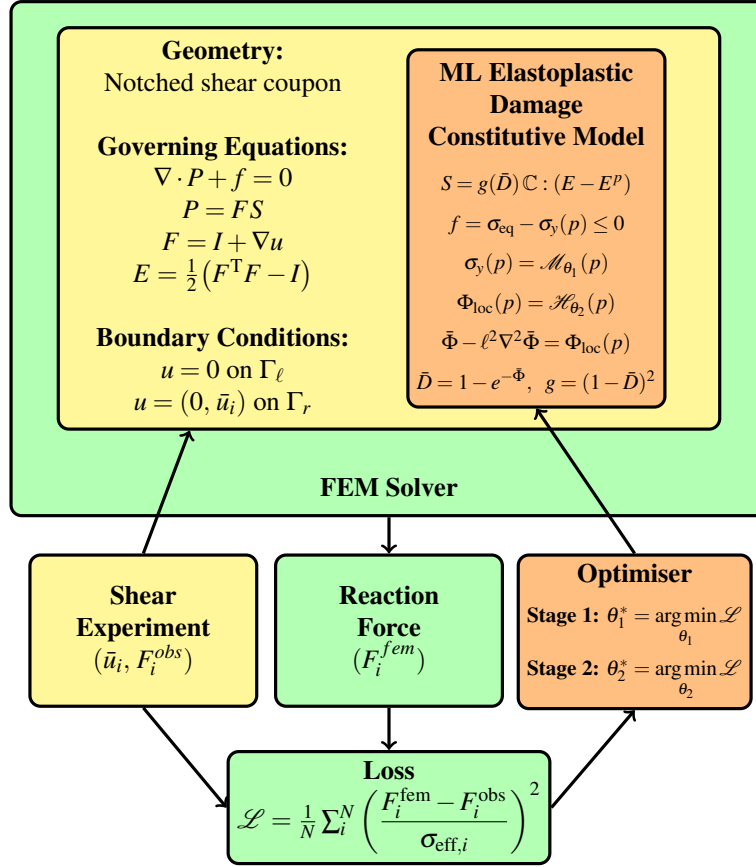
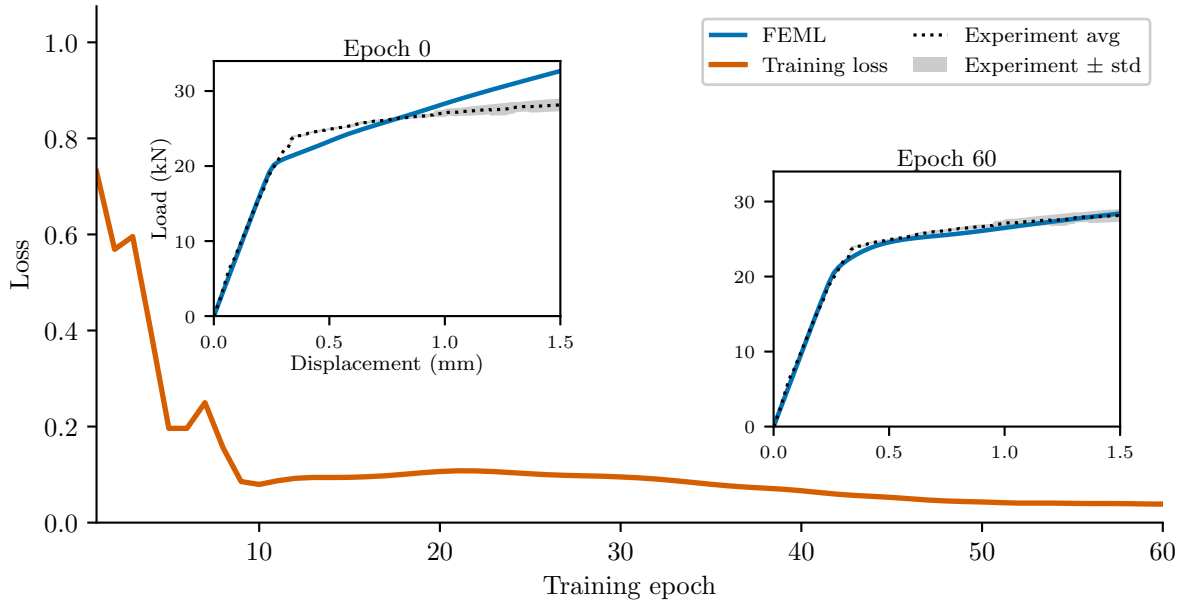
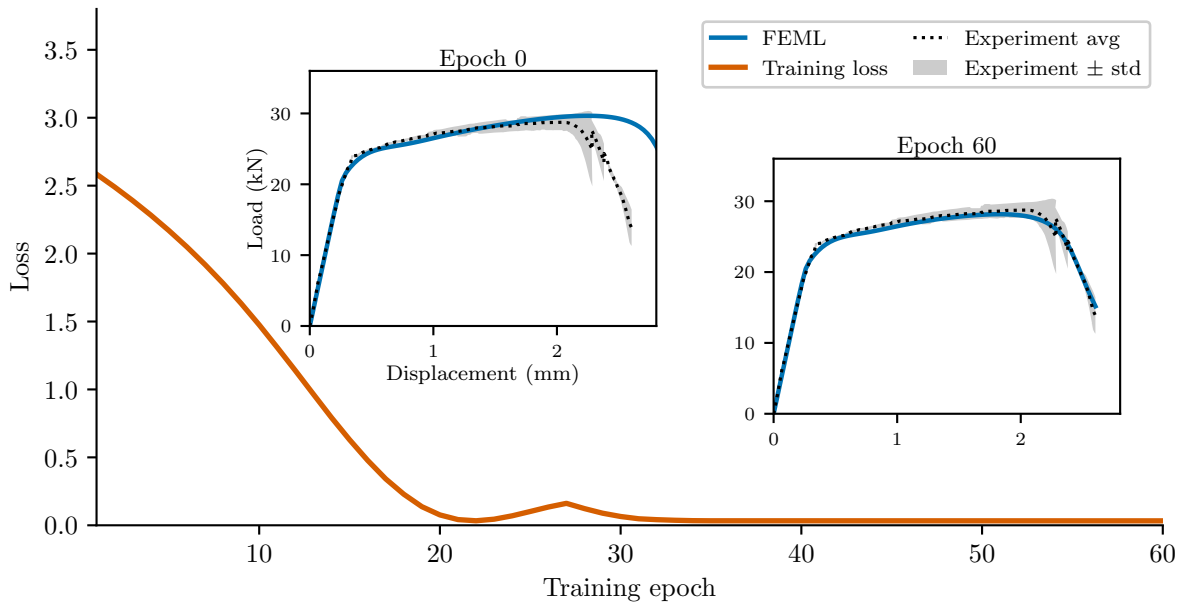


Figure 12. Mathematical definition of the problem and the FEML framework for the shear coupon, in two stages: the hardening law $\sigma_y(p) = \mathcal{M}_{\theta_1}(p)$ and the nonlocal ductile-damage hazard $\Phi_{\text{loc}}(p) = \mathcal{H}_{\theta_2}(p)$, the latter a rectified integral network.

the model does not yet soften and overshoots the experimental peak, whereas the trained rectified integral network reproduces the peak and the post-peak softening tail. The final model tracks the experiment within the specimen-scatter band across the entire curve, achieving a root-mean-square error of 0.54 kN, approximately 1.9% of the peak force.



(a)



(b)

Figure 13. Training the shear-coupon constitutive model. (a) Stage 1, plastic hardening $\sigma_y(p)$ on the rising branch; (b) Stage 2, the rectified-integral-network damage law on the full curve with the hardening operator frozen. Each panel shows the training loss and, in the boxed insets, the predicted (blue) versus measured (dotted) load-displacement response at the initial and final epochs.

Figure 14 shows the fields predicted by the trained model before and after the load peak. As deformation accumulates, the plastic strain and damage concentrate in the ligament between the two notches, forming the localised shear band that drives

the macroscopic softening; the nonlocal length scale gives this band a finite, mesh-objective width. Together, these results demonstrate that FEML can recover a complete elastoplastic–damage description—two learned constitutive operators, including post-peak softening—directly from real laboratory measurements, extending the framework beyond the ground-truth-verified synthetic studies to a genuine experimental benchmark.

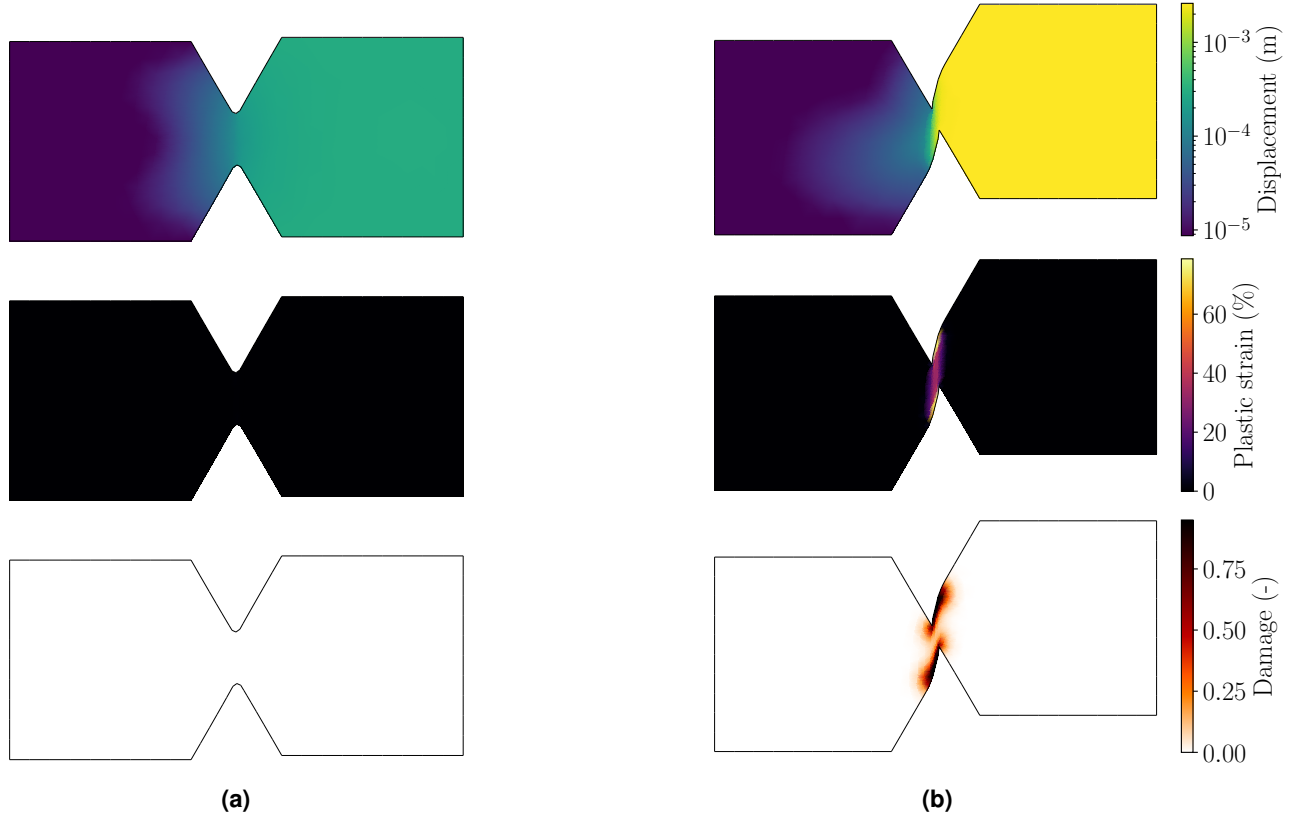


Figure 14. Predicted fields (a) before and (b) after the load peak, shown on the deformed configuration. Top to bottom in each column: displacement magnitude, accumulated plastic strain, and nonlocal damage. Plastic strain and damage localise into the shear band across the ligament between the notches. The deformation has been amplified by four times for visualisation.

1.3 Learning in Transient Thermodynamics Problems

In this section, we showcase the application of the proposed framework to transient thermodynamics problems, focusing on the learning of nonlinear thermal properties. Many materials exhibit temperature-dependent thermal behaviour, which significantly influences heat transfer processes. For instance, rocks, and ceramics respond differently to temperature variations, affecting their thermal performance in geothermal applications, civil engineering applications like bridge thermal expansion analysis, and ceramics, including high-temperature industrial reactors. Capturing these nonlinear thermal properties is essential for accurate simulations and predictive modelling.

The formulation of the transient thermodynamics problem is detailed in Section 3.3. In the following sections, we show how the proposed framework can be used to learn the nonlinear thermal properties of the square plate from temperature measurements.

1.3.1 Learning Thermal Properties from Temperature Measurements

In this example, we consider a transient heat conduction problem on two bodies: a copper disc and a square plate, as shown in Figure 15(a). The copper disc is assumed to have a known constant thermal conductivity, while the square plate has an unknown thermal conductivity that is a nonlinear function of temperature which is parametrised with an ML model.

The two bodies start at room temperature, and then a heat source is applied to the left side of the copper disc, highlighted in dark gray. The heat source generates a fluctuating temperature boundary condition $\bar{T}_i$ that increases in amplitude over time. The temperature field of the square plate is then measured at different times i . Figure 15(b) shows the temperature field at the end of the experiment. The mathematical definition of the problem is illustrated in Figure 16.

The loss function used to train the ML model is defined as:

$$\mathcal{L} = \frac{1}{N} \sum_i^N \frac{|T_i^{\text{fem}} - T_i^{\text{obs}}|}{|T_i^{\text{obs}}|}, \quad (9)$$

where T_i^{fem} represents the temperature field predicted by the FEM solver with the nonlinear ML thermal conductivity model at time step i , and T_i^{obs} is the corresponding synthetic experimental temperature field. The synthetic experimental data is generated by solving the same PDE system with a known nonlinear thermal conductivity model. Both the training and test datasets consist of temperature fields at 12 time steps, but the two datasets are generated from two different synthetic experiments with different temperature boundary conditions. A 2% noise is added to each nodal value of the temperature fields in the synthetic experimental data to represent real data. This noise level is higher than that used in the solid-mechanics examples (1%), and is applied to full-field nodal snapshots at every time step, reflecting a more demanding perturbation.

Figure 17 shows the evolution of the training and test loss, as well as the thermal conductivity profile predicted by the machine learning model embedded in the FEM solver, evaluated at epochs 0, 50 and 100. The dotted line represents the reference thermal conductivity model, which exhibits a nonlinear dependence on temperature, while the blue line shows the corresponding predictions from the ML model. At initialisation (epoch 0), the model fails to capture the correct trend and deviates significantly from the ground truth. By epoch 50, the learned conductivity shows partial agreement, although noticeable discrepancies remain. After 100 epochs, the ML-based model successfully reproduces the nonlinear thermal response, closely matching the reference profile across the entire temperature range. This highlights the model's ability to progressively learn the underlying physical law through training.

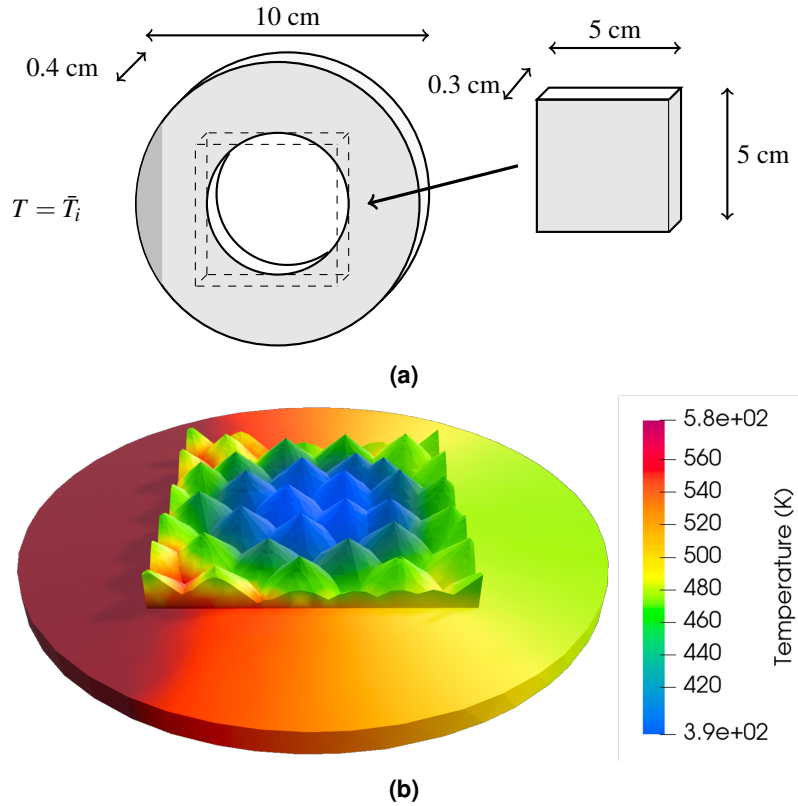


Figure 15. (a) Schematic of the problem showing a disc-shaped copper plate (diameter 10 cm, thickness 0.4 cm, central hole diameter 5 cm) with a square plate of edge 5 cm with an irregular roughness and average thickness 0.3 cm. (b) displays the corresponding temperature distribution.

1.3.2 Learning a Closed-Form Thermal Conductivity Law via Symbolic Regression

Having learned the thermal conductivity operator as a neural network, we now extract a closed-form expression from it using symbolic regression²⁶. This post-processing step converts the trained model into a mathematical formula that can be inspected and interpreted by domain specialists.

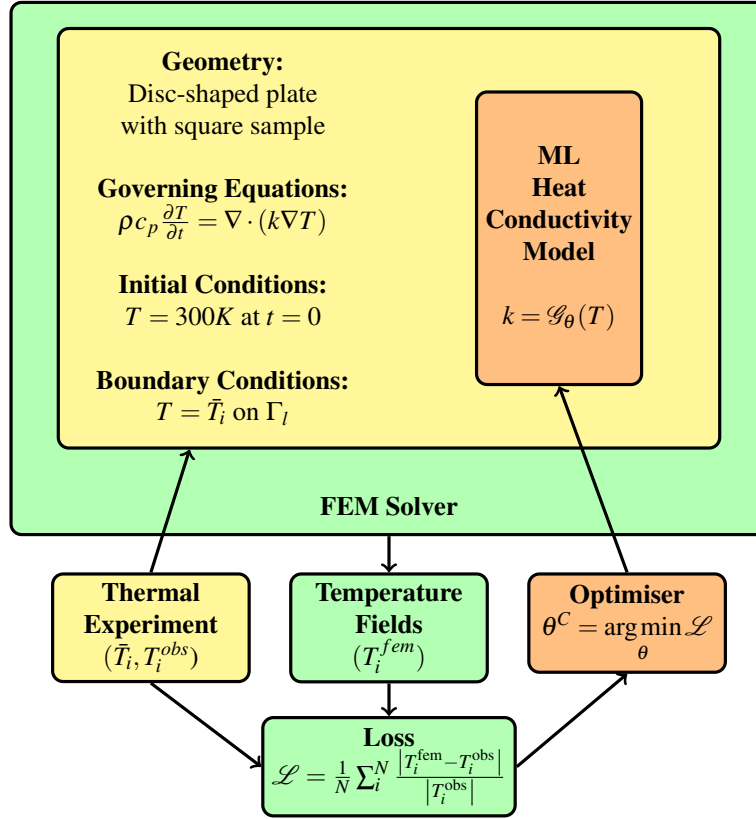


Figure 16. Schematic of the mathematical formulation of the transient heat conduction problem and the proposed framework for learning thermal properties from temperature measurements.

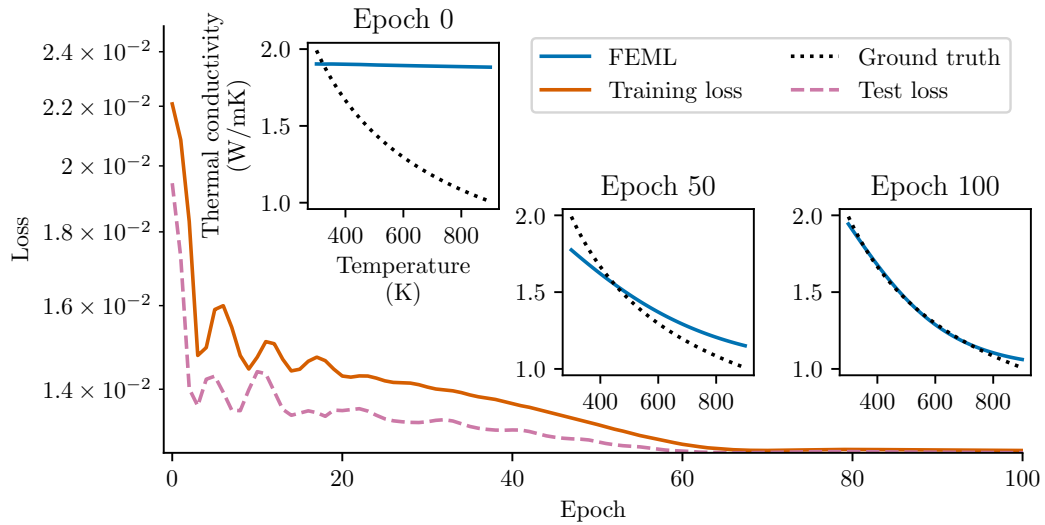


Figure 17. Training and test loss curves are shown alongside the thermal conductivity as a function of temperature for the machine learning model integrated within the FEM solver. Results are presented for training epochs 0, 50 and 100. The dotted line indicates the reference thermal conductivity model (ground truth), while the blue line represents the relationship learned by the ML model. By epoch 100 the model accurately captures the nonlinear dependence of thermal conductivity on temperature observed in the reference model.

We evaluate the trained operator on 500 uniformly spaced temperatures in $[400, 700]$ K—the regime reliably covered during training (see Section 1.3.1)—and fit symbolic expressions to the resulting (T, k_{ML}) data using PySR²⁷. Because thermal conductivity is strictly positive, we augment the mean-squared-error loss with a large penalty for negative predictions, which steers the evolutionary search away from unphysical candidates without constraining the functional form. The resulting Pareto front is then post-filtered: every candidate is evaluated on a dense grid over $[1, 1500]$ K and any expression that produces a negative or non-finite value is discarded. This two-stage strategy—penalised loss during search, global positivity check afterwards—follows standard practice in constrained symbolic regression²⁸ and guarantees $k > 0$ without biasing the discovered expression towards a specific structure such as $\exp(\cdot)$.

Figure 18 compares the ground-truth conductivity, the neural network prediction, and the symbolic expression selected from the filtered Pareto front over $[250, 900]$ K, which includes the extrapolation regime beyond 700 K (the yellow band marks the fitting range). From the candidates that pass the positivity filter we select the one with the highest PySR *score*, defined as the log-loss improvement per unit of added complexity, which balances approximation accuracy against parsimony. The selected expression, of complexity 5, reads:

$$k_{\text{SR}}(T) = \left(\frac{a}{T}\right)^b, \quad (10)$$

with $a = 884.2$ and $b = 0.6547$ (temperature in K, conductivity in $\text{W m}^{-1} \text{K}^{-1}$). This pure power-law form is noteworthy because the ground truth rewrites as $k = (911.6/T)^{0.62}$: the physical signal captured by FEML was rich enough for symbolic regression to recover, without any prior knowledge of the underlying law, essentially the same $(C/T)^\alpha$ structure—with a within 3% of the exact constant and b within 6% of the exact exponent. The expression achieves $R^2 = 0.997$ against the ground truth on the fitting range $[400, 700]$ K and $R^2 = 0.886$ in the extrapolation region beyond 700 K. The inset of Figure 18 displays the Pareto front of all candidate expressions returned by the symbolic regression search, plotting mean squared error on a logarithmic scale against expression complexity. The MSE decreases by roughly three orders of magnitude between complexities 3 and 5, after which the front plateaus: expressions of complexity 7 through 20 offer only marginal reductions in error. The blue triangle marks the selected expression (complexity 5), which sits at the knee of the Pareto front where accuracy gains per additional unit of complexity become negligible.

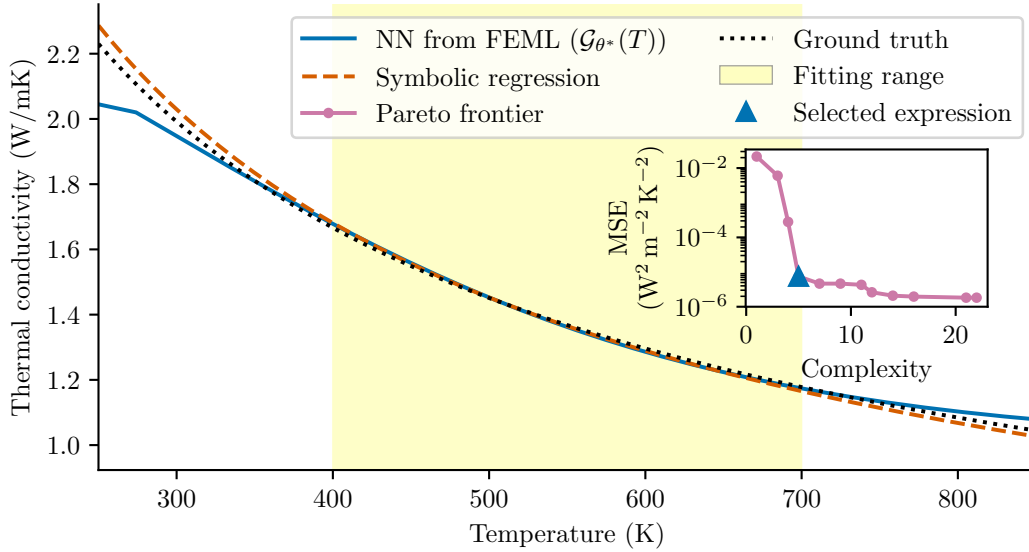


Figure 18. Symbolic regression of the learned thermal conductivity operator. A penalised loss steers the search away from negative predictions and a post-hoc positivity filter over $[1, 1500]$ K ensures $k > 0$ globally. The main axes compare the thermal conductivity as a function of temperature: ground truth (dotted black), NN from FEML $\mathcal{G}_{\theta^*}(T)$ (solid blue), and the selected symbolic regression expression (dashed vermilion). The yellow band marks the fitting range $[400, 700]$ K. The inset shows the Pareto front of candidate expressions (mean squared error versus complexity); the blue triangle marks the selected expression (complexity 5), chosen as the highest-scoring candidate among those that satisfy the positivity constraint.

2 Discussion and Conclusions

We have introduced FEML, a fully differentiable finite element–based machine learning framework for discovering missing physics in systems governed by partial differential equations. The key idea is to retain the known governing equations in their standard finite element (FEM) form, while representing the unknown physical relationships as trainable operators embedded within the variational formulation. By differentiating end-to-end through both the FEM solver and the machine learning components, FEML enables the identification of internal operators—such as constitutive or transport laws—from indirect, experimentally accessible quantities. In contrast to surrogate models that learn configuration-specific PDE solutions, the learned operator in FEML is defined on function spaces and can be reused across geometries, boundary conditions, dimensions, and discretisations.

A central ingredient of the framework is the use of structure-preserving operator networks (SPONs) to parameterise the missing-physics operator. By operating on the degrees of freedom of finite element spaces, SPONs preserve key properties of the continuous operators at the discrete level, and come with theoretical guarantees. SPON models offer zero-shot cross-discretisation capabilities, which we demonstrated on our numerical experiments: once trained, the operator can be evaluated on meshes and finite element spaces different from those used during training, without additional retraining.

We assessed FEML on a sequence of solid mechanics problems in which the hidden operator is a nonlinear constitutive relation for an unknown material. The constitutive law was discovered progressively in two stages. In the first stage (displacement-controlled uniaxial test), FEML recovered a nonlinear elastic softening law from a noisy and extremely sparse dataset: just six force–displacement values, perturbed with 1% noise. The loading was kept within the elastic regime, so only the strain-dependent Young’s modulus $E(\kappa)$ was active. Despite having access only to global load measurements, and no direct stress information, the learned operator reconstructed the full elastic response closely matching the reference behaviour. This illustrates both the data efficiency of the framework and the advantage of exploiting equilibrium constraints through the embedded FEM solver.

In the second stage (load-controlled Brazilian disc test), the learned elastic operator was frozen and the framework leveraged full-field displacement data to identify the plastic hardening law $\sigma_y(p)$ of the same material. The learned yield-stress evolution reproduced complex displacement fields under indirect loading when trained on noisy nodal displacement fields at 15 load levels. This sequential discovery strategy—learning elastic, then plastic, behaviour from progressively richer data—demonstrates that FEML can compose frozen and trainable operators within a single differentiable framework to incrementally build up a complete constitutive description.

We note that the sequential strategy adopted here is not the only option available within FEML. Because the framework supports end-to-end differentiability through arbitrary compositions of PDE solvers and ML operators, one could alternatively define a joint loss that combines data from both experiments—the force residuals from the uniaxial test and the displacement-field residuals from the Brazilian disc test—and train the elastic and plastic operators concurrently. Joint training may be advantageous when the phenomena of interest are strongly coupled or when data from a single experiment type are insufficient to uniquely identify either operator in isolation. The sequential approach was preferred here because it decouples the learning tasks, simplifies the optimisation landscape, and mirrors the staged experimental workflow commonly adopted in practice; however, the joint formulation remains a natural extension enabled by the framework’s differentiable architecture.

We then combined both pretrained operators— $E(\kappa)$ and $\sigma_y(p)$ —into a foundation elastoplastic constitutive model and applied it in a zero-shot fashion to a three-dimensional cylindrical rod with a transverse keyhole-shaped through-slot under torsion. Without any retraining, the learned operators produced displacement and stress fields in close agreement with the reference solution. This level of accuracy is noteworthy given that the operators were trained on noisy data from different two-dimensional mechanical tests and then transferred to a 3D geometry, with different loading and boundary conditions. This result illustrates a main advantage of operator-level learning over solution-level surrogates: once the hidden physics has been identified, the same operators can be deployed as reusable components in new scenarios, rather than being restricted to the configuration on which they were trained.

As a demonstration of real-world applicability, we moved from synthetic to *real* experimental data, applying the framework to a benchmark shear-coupon test on a titanium alloy. This example extends the discovered description beyond elasticity and hardening to ductile damage: a plastic-hardening law and a ductile-damage hazard are represented by two learned neural operators—both monotone networks built from the integral of a non-negative activation, with the activation matched to each law (a smooth rise for hardening, a hard onset followed by accelerating growth for damage)—and trained end-to-end through the adjoint within a finite-strain formulation. Because the underlying constitutive law is unknown for real data, the damage operator is regularised by a nonlocal length scale that renders the softening band mesh-objective, and success is judged by agreement with the measured response. The trained model reproduces the full shear load–displacement curve—elastic loading, yield, plateau, peak, and post-peak softening—to within the experimental specimen-to-specimen scatter, with the plastic strain and damage localising into a shear band across the notched ligament. This demonstrates that FEML composes multiple learned constitutive operators and recovers a complete elastoplastic–damage description directly from real laboratory measurements,

not only from synthetic data.

To demonstrate FEML applications beyond quasi-static solid mechanics, we apply the framework to a transient heat-conduction problem in which the missing physics is an unknown temperature-dependent thermal conductivity. The operator was trained from noisy temperature fields (with 2% perturbations) measured on a heterogeneous, coupled 3D geometry comprising a copper disc and an irregular square plate. The learned thermal law accurately reproduced the underlying nonlinear conductivity profile and generalised to a test experiment with different temperature boundary conditions. This example demonstrates that the framework extends naturally to time-dependent problems and heterogeneous materials, and that it can accommodate realistic levels of measurement noise in complex three-dimensional configurations. Furthermore, we demonstrated that a compact closed-form expression can be learned from the neural operator via symbolic regression (Section 1.3.2). The selected expression achieves $R^2 > 0.99$ on the fitting range and, notably, recovers the same pure power-law structure $(C/T)^\alpha$ as the ground truth, with the fitted constants within 3% and 6% of their exact values—even though no prior knowledge of the underlying functional form was provided to the search. This indicates that the physical signal encoded in the FEML-trained neural network is faithful enough for the true analytical law to be extracted automatically. Taken together, these results demonstrate that the combined FEML–symbolic-regression framework can recover closed-form physical laws from indirect, noisy measurements without presupposing the functional form of the underlying relation.

The present work also suggests several opportunities and limitations that warrant further investigation. First, although end-to-end differentiability through a mature FEM stack enables flexible model discovery, it comes at a computational cost: each training epoch requires both a forward FEM solve and an adjoint solve for gradient computation, and this cost is repeated for every load or time step in the training dataset. Across all examples, end-to-end training takes from a few minutes for the simplest case to a few hours for the most demanding, on a single laptop—a MacBook Air with a 10-core Apple M4 chip and 32 GB of unified memory—with the cost set mainly by the mesh resolution, the number of load or time steps evaluated per epoch, and the complexity of the constitutive update.

The displacement-controlled example uses a coarse 6×6 rectangular mesh with cubic elements and a single ML model, resulting in the shortest training time. The load-controlled Brazilian disc example is more expensive owing to a finer disc mesh, the J2 elastoplastic radial-return algorithm, and the evaluation of full-field displacement errors at each epoch. The transient heat conduction example operates on a 3D tetrahedral mesh with 12 time steps per epoch, which increases the per-epoch cost. We note that the computational overhead of the ML operator during forward solves is negligible: performance benchmarks show that the difference in solve time between a standard FEM and an ML-augmented FEM solve is less than 1.3% across all mesh sizes tested; the training cost is dominated by the adjoint solves required for backpropagation. Efficient solvers, reduced-order models, and mixed-precision or multi-fidelity training strategies will be important to scale FEML to field-scale applications. A summary of all ML hyper-parameters and a sensitivity analysis with respect to training epochs are provided in Section 3.4; the analysis shows that the learned operators stabilise well before final convergence and that training and test losses track each other closely, indicating low sensitivity to the specific hyper-parameter choices. Second, identifiability issues can arise when multiple operators produce similar observables. In the FEML setting, three features mitigate this risk: (i) the operator is embedded inside a PDE whose governing equations and boundary conditions must be satisfied at every spatial point simultaneously and across multiple load or time steps (e.g., momentum balance in the solid-mechanics examples, energy conservation in the thermal example), imposing far more constraints than learning from paired input–output data alone; (ii) the specific neural-network parameterisation within the SPON structure restricts the hypothesis class to physically admissible mappings; in the constitutive examples, the invariant-based inputs, isotropic tensor basis, and positive activation functions (e.g., SoftPlus) enforce isotropy, frame indifference, and positivity of moduli by construction, eliminating non-physical parametrisations; and (iii) spatially heterogeneous loading (e.g., the Brazilian disc test) activates diverse local strain states from a single experiment, densely sampling the operator’s input domain. The zero-shot generalisation result, where constitutive operators trained on 2D tests are deployed on a three-dimensional cylindrical rod with a keyhole through-slot under torsion, provides empirical evidence that the learned operators capture the true material law rather than merely fitting the training configuration. Nevertheless, formal identifiability guarantees are not established in this work, and in general, addressing non-uniqueness will require careful experiment design, regularisation informed by prior knowledge, and possibly multi-modal data.

All experiments except the shear-coupon study (Section 1.2.4) use synthetic data generated from the same finite element solver (Firedrake). This choice is deliberate: synthetic data provides an exact ground-truth operator against which the learned model can be quantitatively validated, something that is not possible with real experimental data, where the true constitutive or transport law is unknown. We note, however, that several structural features of the framework mitigate the risk of overfitting to solver-specific numerical artefacts. The ML model learns a local, pointwise material law (e.g., strain invariants to Lamé parameters, or temperature to conductivity) and has no access to the mesh topology, element shape functions, or linear-algebra internals of the solver. Furthermore, the synthetic examples presented span fundamentally different governing equations, quasi-static momentum balance (an elliptic vector system) and transient heat conduction (a parabolic scalar equation), with different discretisations, element types, and time-stepping schemes; there is no common “numerical behaviour” across these problems to

which the ML model could overfit. The zero-shot generalisation experiment provides additional evidence: constitutive operators trained on 2D meshes under uniaxial compression and diametral loading were deployed on a 3D tetrahedral mesh under torsion. Had the model memorised solver-specific artefacts, this cross-dimension, cross-geometry, cross-loading transfer would have failed. We have also taken a first step towards validation on real measurements: the shear-coupon study (Section 1.2.4) learns both a plastic-hardening and a ductile-damage operator directly from a benchmark experiment, reproducing the measured response—including post-peak softening—to within the experimental specimen-to-specimen scatter. Broader validation across materials, geometries, and loading regimes remains a natural next step.

We note that a direct quantitative comparison between FEML and solution-level surrogate methods such as FNO or PINO is not applicable, because the two classes of methods address fundamentally different learning problems and produce different types of output. Solution-level surrogates learn the full mapping from PDE inputs (e.g., forcing, boundary conditions) to PDE solutions; they are designed for fast evaluation within a training distribution but do not extract a reusable, configuration-independent operator. FEML, by contrast, learns the hidden operator itself, which can then be ported across geometries, boundary conditions, dimensions, and discretisations—as demonstrated by the zero-shot 2D→3D transfer experiment in Section 1.2.3. Similarly, while inverse-PINN formulations¹³ can in principle be set up for operator-discovery problems, they enforce PDE residuals at collocation points (strong form) rather than through the variational formulation used by FEML, and do not employ structure-preserving operator networks—the key ingredient enabling zero-shot cross-discretisation generalisation. These methods are thus best understood as complementary rather than competing: surrogates deliver fast inference for parameterised PDE families, while FEML discovers the underlying physical law.

Future work that goes beyond direct physics discovery, will include the use of FEML in a surrogate-modelling, where the embedded ML operator emulates expensive high-order coupling terms in a full-order PDE model. In such a workflow, high-fidelity simulations provide training data for the operator, which then replaces the costly components in subsequent simulations. This strategy has the potential to deliver reduced-order models that retain the fidelity of the original FEM formulation while significantly lowering computational cost, with applications ranging from nonlinear constitutive modelling and fracture to multiphase flow and thermomechanical coupling.

Although the current implementation is built on a finite element solver (Firedrake), the conceptual framework is not restricted to FEM. The underlying mathematical formulation, i.e. minimising a loss where the forward state satisfies a discrete residual containing a trainable operator, applies equally to finite-volume, finite-difference, or other discretisation methods, provided that (i) the solver supports adjoint-based or algorithmic differentiation for computing gradients through the forward solve, and (ii) its differentiation system can be composed with the computational graph of the ML library to enable end-to-end gradient flow. As discussed in the Introduction, differentiable non-FEM solvers already exist; however, certain features of the current framework, in particular, structure-preserving operator networks (SPONs) and zero-shot super-resolution, exploit the finite element function-space structure and would require adaptation for alternative discretisations. Importantly, the learned operator itself is solver-independent: since the ML model learns a local, pointwise material law, it can in principle be deployed inside any compatible solver at inference time.

In summary, FEML provides a general and flexible route to combine physics-based finite element solvers with data-driven operator learning. By preserving known physics, learning only the missing relations, and respecting the underlying function-space structure, the framework yields data-efficient and interpretable operators that can be ported across problems, geometries, and discretisations. The solid-mechanics examples presented here demonstrate sequential discovery of elastic and plastic behaviour for the same material, with the learned operators composed into a foundation model that generalises zero-shot to a three-dimensional torsion problem. On real laboratory data, the framework composed two learned operators—a plastic-hardening law and a ductile-damage hazard—and reproduced the measured shear-coupon response, including post-peak softening, to within the experimental specimen-to-specimen scatter. The thermal conductivity example further shows that closed-form expressions can be learned from the neural operators via symbolic regression, completing the framework from indirect measurements to human-interpretable physical laws. We expect this combination of differentiable PDE solvers and structure-preserving operator networks to underpin a new class of predictive tools for computational solid mechanics, thermodynamics, and, more broadly, for scientific and engineering applications in which incomplete physical knowledge must be reconciled with limited but informative data.

3 Methods

3.1 Differentiable Coupling of Firedrake and ML Frameworks

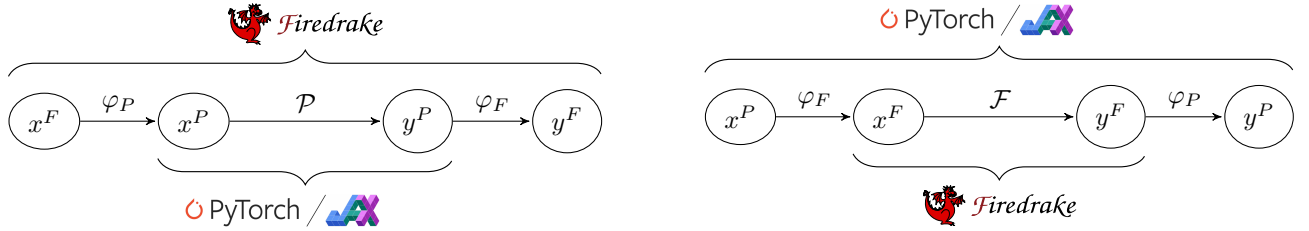
Learning the operator $\mathcal{G}_\theta$ from indirect observations requires end-to-end gradient flow through the coupled FEM–ML system. This section describes how that gradient flow is realised; full details are given in².

Computational graph composition. The key idea is to view the coupled FEM–ML simulation as a single directed acyclic graph (DAG). The coupling strategy introduced in² partitions this DAG into two sub-graphs: Firedrake-based operations are

differentiated by *pyadjoint*, whereas ML operations are handled by the native AD engine of the ML library (*autograd* for PyTorch¹⁷, or *grad* for JAX¹⁸). This partitioning retains the full capabilities and performance of each framework.

The coupling operates in two directions (Figures 19a and 19b):

1. *Embedding ML models into PDE systems* (Figure 19a). An ML model, such as a PyTorch neural network, is exposed as a symbolic operator within UFL²⁹, the domain-specific language that Firedrake uses to express variational forms. A dedicated `ml_operator` interface represents the ML model as a UFL linear form with k coefficient operands. Symbolic UFL operations such as `derivative`, `action`, and `adjoint` are then transparently dispatched to the ML framework's AD engine. In practice, this means that the derivatives $\frac{\partial N}{\partial u}$ (required for Newton solves) and $\frac{\partial N}{\partial \theta}$ (required for training) are computed by PyTorch or JAX without any manual implementation of derivative expressions.
2. *Embedding PDE solvers into ML frameworks* (Figure 19b). A Firedrake solve is registered as a custom differentiable operator in `torch.autograd` (or equivalently in JAX). During the forward pass, the PDE is solved by Firedrake; during the backward pass, *pyadjoint* evaluates the adjoint model (13)–(14). Lightweight casting operations φ_F and φ_P convert between ML tensors and Firedrake functions, handling the different data representations of the two frameworks². The resulting composite operator makes the PDE solution, together with its derivative information, a first-class node in the ML computational graph, so that standard optimisers such as Adam can update θ through end-to-end backpropagation.



(a) Embedding ML models into the Firedrake computational graph. $\mathcal{P}$ denotes ML operations; P and F denote PyTorch/JAX and Firedrake variables, respectively. Adapted from².

(b) Embedding Firedrake operations into the PyTorch/JAX computational graph. $\mathcal{F}$ denotes Firedrake operations. Adapted from².

Figure 19. Differentiable coupling between Firedrake and ML frameworks. In both directions, the casting operations φ_F and φ_P convert between PyTorch/JAX tensors and Firedrake functions. Adapted from².

Adjoint-based differentiation through the PDE solve. Consider the following concrete illustration of how backpropagation is realised when a variational problem appears as a node in the coupled DAG. The PDE solution u_θ depends implicitly on the ML parameters θ through $\mathcal{G}_\theta$, and computing $\frac{du_\theta}{d\theta}$ by differentiating through every Newton iteration would be prohibitively expensive. Instead, the gradient is obtained via the *adjoint method*. Let V and M be Hilbert spaces, and let $u \in V$ be the solution of the parametrised PDE

$$F(u, m; v) = 0 \quad \forall v \in V, \quad (11)$$

where $m \in M$ denotes the control (in our context, the ML parameters θ or any other quantity that $\mathcal{G}_\theta$ depends on). Under the assumptions that F is continuously Fréchet differentiable and that the linearised operator $\frac{\partial F}{\partial u}$ is non-singular, the implicit function theorem ensures existence and differentiability of the solution map $u(\cdot): M \rightarrow V$ ³⁰. Implicit differentiation of $F(u(m), m; v) = 0$ with respect to m then gives

$$\frac{du}{dm} = - \left(\frac{\partial F}{\partial u} \right)^{-1} \frac{\partial F}{\partial m}. \quad (12)$$

Backpropagation through the PDE solve is realised by the adjoint model of $u(m)$, which takes the form

$$\mathcal{J}_{u,m}^*(w) = - \frac{\partial F^*}{\partial m} \lambda, \quad \forall w \in V^*, \quad (13)$$

where $\lambda \in V$ is the solution of the *adjoint equation*

$$\frac{\partial F^*}{\partial u} \lambda = w. \quad (14)$$

In FEM, this adjoint solve is performed automatically by the *pyadjoint* package³¹, which tapes the forward Firedrake computation and derives the corresponding adjoint system without user intervention.

End-to-end gradient flow for training. As a natural consequence of combining both embedding directions, the complete gradient chain required for training in FEML is assembled automatically. Given a loss $\mathcal{L}(u_\theta, u^{\text{obs}})$, the gradient with respect to the ML parameters θ is computed as follows: (i) the ML framework differentiates $\mathcal{L}$ with respect to u_θ ; (ii) *pyadjoint* solves the adjoint equation (14) to propagate gradients through the PDE solve; and (iii) the ML framework differentiates through $\mathcal{G}_\theta$ to obtain $\frac{d\mathcal{L}}{d\theta}$. This three-stage chain requires no manual derivation of adjoint equations or gradient expressions. The implementation is built on Firedrake¹⁵, which is publicly available, and uses its native *pyadjoint* integration for adjoint computations. All examples presented in this work can be reproduced using Firedrake.

3.2 Solid mechanics problem formulation

All solid-mechanics examples are quasi-static, but they span two kinematic regimes. The displacement-controlled (Section 3.2.1), load-controlled (Section 3.2.2), and zero-shot torsion (Section 3.2.3) examples undergo small strains and rotations and are modelled with infinitesimal, small-displacement kinematics; the shear-coupon example (Section 3.2.4) develops large rotations within its ligament and is modelled with a finite-strain, total-Lagrangian formulation. The momentum balance and the strain and stress measures differ between the two regimes, but the way a learnable operator is embedded in the constitutive law does not.

In the small-displacement regime, equilibrium is expressed in the (undeformed) configuration through the balance of linear momentum,

$$\nabla \cdot \sigma + f = 0, \quad (15)$$

where σ (N/m²) is the Cauchy stress tensor and f (N/m³) denotes body forces, taken to be zero in all examples presented here.

The infinitesimal strain tensor is defined by the symmetric gradient of the displacement field:

$$\varepsilon = \frac{1}{2} (\nabla u + (\nabla u)^T), \quad (16)$$

where u (m) is the displacement field and ε is dimensionless.

When strains or rotations are no longer small, this linearisation is inadequate and a finite-strain, total-Lagrangian description is used instead. Equilibrium is then posed in the reference (undeformed) configuration,

$$\nabla_0 \cdot P + f = 0, \quad (17)$$

where P is the first Piola–Kirchhoff stress and ∇_0 the gradient taken with respect to the reference coordinates. Deformation is measured by the deformation gradient and the work-conjugate Green–Lagrange strain,

$$F = I + \nabla u, \quad E = \frac{1}{2} (F^T F - I), \quad (18)$$

and the constitutive law relates the second Piola–Kirchhoff stress S to E , with $P = FS$. The small-displacement equations are the limiting case of this description: as strains and rotations vanish $F \rightarrow I$, so that $E \rightarrow \varepsilon$ and the stress measures coincide, $P \rightarrow S \rightarrow \sigma$, recovering Eqs. (15)–(16).

In either regime the system is closed by a constitutive relation between a strain measure and its work-conjugate stress. In a classical setting this relation is fully specified by known material laws; here, one or more material functions are unknown and are replaced by learnable operators. In its most general form the ML constitutive model is written as

$$\sigma = \mathcal{G}_\theta(\varepsilon) \quad (\text{small strain}), \quad S = \mathcal{G}_\theta(E) \quad (\text{finite strain}), \quad (19)$$

where $\mathcal{G}_\theta$ denotes the constitutive operator with learnable parameters θ , acting on the strain measure appropriate to the regime. Crucially, $\mathcal{G}_\theta$ is not a black-box mapping from strain to stress; it retains the known physical structure of the constitutive law and embeds a learnable neural network only in place of the specific material function that is unknown. This structure-preserving embedding is identical in both regimes, so the same learning strategy applies whether the kinematics are linearised or finite.

The following subsections detail the problem setup, ground-truth material law, and ML model architecture for each experiment: in the displacement-controlled experiment (Section 3.2.1), the sole unknown is the nonlinear elastic modulus,

which is expressed as a neural network learning the exponential softening behavior of the ground-truth law. In the load-controlled experiment (Section 3.2.2), the learned elastic operator is frozen and coupled with a second neural network that parameterises the yield-stress function $\sigma_y(p)$ to learn the ground-truth Voce hardening law. The two pretrained operators are then composed into a foundation constitutive model and deployed zero-shot on a three-dimensional torsion problem (Section 3.2.3). Finally, the shear-coupon experiment (Section 3.2.4) learns a plastic-hardening and a ductile-damage operator from real data within a finite-strain formulation; as there is no ground-truth law, that section describes the forward model and the two network architectures rather than a reference material law.

3.2.1 Simulating Displacement-Controlled Uniaxial Experiments

The specimen simulated in Section 1.2.1 is discretised with a triangular mesh, as shown in Figure 20. The displacement field is approximated with cubic polynomial basis functions.

Ground Truth The ground truth constitutive law describes a closed-cell polymeric foam: the Poisson's ratio is fixed to a value of $\nu = 0.3$, while the Young's modulus softens under compressive volumetric strain:

$$E(\kappa) = E_\infty + (E_0 - E_\infty) e^{-\kappa/\kappa_0}, \quad (20)$$

where $\kappa = \langle -\text{tr}(\boldsymbol{\varepsilon}) \rangle$ is the Macaulay bracket of the compressive volumetric strain, $E_0 = 145$ MPa is the initial (undeformed) modulus, $E_\infty = 57.3$ MPa is the fully softened modulus, and $\kappa_0 = 0.008$ controls the rate of softening.

ML Model Architecture In this example only the Young's modulus is unknown; the Poisson's ratio is treated as a known constant. The constitutive operator of Eq. (19) therefore takes the form:

$$\boldsymbol{\sigma} = \mathcal{G}_\theta(\boldsymbol{\varepsilon}) = \lambda(\mathcal{N}_\theta(I_1), \nu) \text{tr}(\boldsymbol{\varepsilon}) \mathbf{I} + 2\mu(\mathcal{N}_\theta(I_1), \nu) \boldsymbol{\varepsilon}, \quad (21)$$

where $\mathcal{N}_\theta(I_1)$ is a single MLP that takes the first strain invariant $I_1 = \text{tr}(\boldsymbol{\varepsilon})$ as input and outputs a positive scalar replacing the Young's modulus, $\nu = 0.3$ is the known Poisson's ratio, and the Lamé coefficients are computed from these quantities via the isotropic linear elastic constitutive relations:

$$\lambda = \frac{\mathcal{N}_\theta(I_1) \nu}{(1 + \nu)(1 - 2\nu)}, \quad \mu = \frac{\mathcal{N}_\theta(I_1)}{2(1 + \nu)}. \quad (22)$$

The MLP has two hidden layers, each containing thirty neurons, with SiLU and SoftPlus activation functions. A single MLP outputs E from I_1 , and the Lamé parameters are computed via the standard isotropic relations, guaranteeing frame indifference, isotropy, and positivity of the elastic moduli.

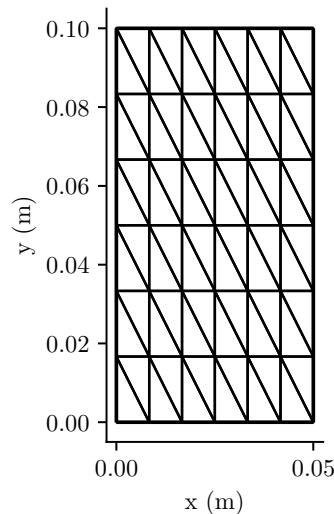


Figure 20. Computational mesh for the displacement-controlled uniaxial test.

3.2.2 Simulating Load-Controlled Brazilian Disc Experiments

The specimen simulated in Section 1.2.2 is a discretised with a triangular mesh, as shown in Figure 21. The displacement field is approximated with linear polynomial basis functions.

Ground Truth The ground truth constitutive law combines the elastic softening modulus of Eq. (20) with J2 elastoplasticity and Voce isotropic hardening. The Poisson’s ratio is fixed at $\nu = 0.3$ and the elastic modulus $E(\kappa)$ is the same foam law described in Section 3.2.1. The Lamé parameters are computed from E and ν via the isotropic linear elastic constitutive relations. The yield stress evolves according to Voce hardening:

$$\sigma_y(p) = \sigma_{y0} + R_\infty \left(1 - e^{-bp}\right), \quad (23)$$

where p is the accumulated plastic strain, $\sigma_{y0} = 2.354$ MPa is the initial yield stress, $R_\infty = 0.9$ MPa is the saturation hardening, and $b = 25$ controls the rate of hardening. Given the total strain tensor ε , the stress is computed from the elastic strain as

$$\sigma = \mathbb{C}(\kappa) : (\varepsilon - \varepsilon^p), \quad f(\sigma, p) = \sigma_{eq}(\sigma) - \sigma_y(p) \leq 0, \quad (24)$$

where $\varepsilon = \varepsilon^e + \varepsilon^p$ is the additive decomposition of the (infinitesimal) total strain into elastic and plastic parts, σ_{eq} is the von Mises equivalent stress, and f is the yield function, so that $\sigma_y(p)$ sets the current admissible stress. The plastic strain ε^p obeys the associative flow rule, and the accumulated (equivalent) plastic strain p —the scalar internal variable that drives hardening—is the time integral of its rate,

$$\dot{\varepsilon}^p = \dot{p} \frac{\partial f}{\partial \sigma}, \quad \dot{p} = \sqrt{\frac{2}{3}} \dot{\varepsilon}^p : \dot{\varepsilon}^p, \quad p(t) = \int_0^t \dot{p} dt', \quad (25)$$

subject to the Karush–Kuhn–Tucker loading/unloading conditions $\dot{p} \geq 0$, $f \leq 0$, $\dot{p} f = 0$. The J2 radial-return mapping enforces these conditions at each quadrature point.

ML Model Architecture The pretrained elastic operator $E(\kappa)$ from Section 1.2.1 is embedded in the solver with its weights frozen. Because J2 plastic flow is isochoric ($\text{tr}(\varepsilon^p) = 0$), the total volumetric strain equals the elastic volumetric strain, so $\kappa = \langle -\text{tr}(\varepsilon) \rangle$ can be evaluated directly from the displacement field without requiring the elastic–plastic decomposition. The only trainable component is the yield-stress function $\sigma_y(p)$, parameterised by the integral monotone neural network described below.

The yield-stress function $\sigma_y(p)$ must be non-decreasing with respect to the accumulated plastic strain p to ensure thermodynamic consistency. A standard approach would be to train a neural network to output $\sigma_y(p)$ directly and add a penalty term to the loss to discourage monotonicity violations. However, a penalty only *encourages* the desired property; it does not guarantee it. Instead, we encode monotonicity directly in the network architecture via an integral formulation that *enforces* it by construction.

The key idea is to let the network learn not the yield stress itself, but the local *hardening rate*—how fast the yield stress grows with plastic strain—and then integrate that rate to obtain the yield stress:

$$\sigma_y(p) = \sigma_{y0} + \int_0^p \text{softplus}(\text{MLP}(s)) ds, \quad (26)$$

where σ_{y0} is a trainable scalar representing the initial yield stress at $p = 0$, and $\text{MLP}(s)$ is a multi-layer perceptron with a single hidden layer. Because $\text{softplus}(x) = \log(1 + e^x) > 0$ for all x , the integrand is strictly positive everywhere, and therefore the accumulated integral can only increase with p . Formally:

$$\frac{d\sigma_y}{dp} = \text{softplus}(\text{MLP}(p)) > 0, \quad (27)$$

so $\sigma_y(p)$ is monotonically increasing by construction, regardless of the learned network weights. This is stronger than a soft penalty: the hypothesis space is restricted to physically admissible hardening laws.

Because the integrand contains a neural network, the integral in Eq. (26) generally has no closed form. We approximate it numerically using Gauss–Legendre quadrature:

$$\int_0^p \text{softplus}(\text{MLP}(s)) ds \approx \sum_{i=1}^{n_{\text{quad}}} w_i \text{softplus}(\text{MLP}(s_i)), \quad (28)$$

where $\{s_i, w_i\}$ are the quadrature nodes and weights. Standard Gauss–Legendre nodes are defined on the reference interval $[-1, 1]$; at each evaluation they are affinely mapped to the current interval $[0, p]$, which varies from one material point and

load step to another. Because the integrand is smooth, Gauss–Legendre quadrature converges rapidly; we use $n_{\text{quad}} = 16$ points, which provides sufficient accuracy. Each evaluation of $\sigma_y(p)$ therefore requires n_{quad} forward passes through the (small) MLP—a negligible cost compared with the PDE solve.

In J2 plasticity, the radial-return algorithm used in the plastic corrector step requires not only $\sigma_y(p)$ but also its derivative with respect to p , the hardening tangent $H(p) := d\sigma_y/dp$. A key advantage of the integral formulation is that, by the fundamental theorem of calculus, this tangent is simply the integrand evaluated at the upper limit:

$$H(p) = \text{softplus}(\text{MLP}(p)). \quad (29)$$

This requires only a single MLP evaluation at p —no additional integration, finite differencing, or automatic differentiation through the quadrature routine is needed. The resulting tangent is both exact (it follows from the analytic definition of the model) and inexpensive, providing a smooth and accurate input to the radial-return solver and thereby contributing to the stability of the local plasticity update.

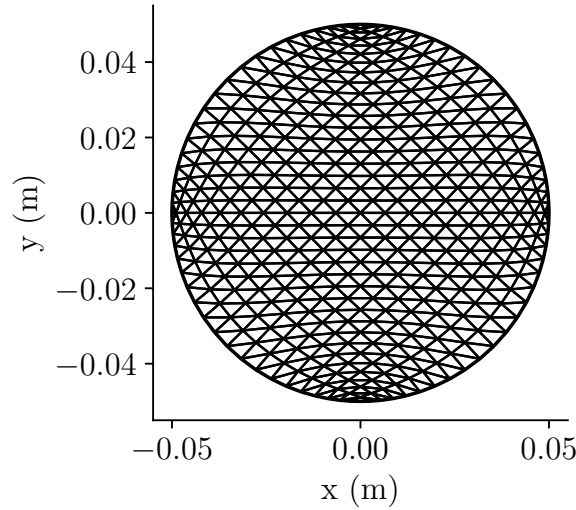


Figure 21. Computational mesh for the load-controlled Brazilian disc test.

3.2.3 Simulating Torsional Behaviour in a Slotted Cylindrical Rod

The cylindrical rod simulated in Section 1.2.3 is discretised with the tetrahedral mesh shown in Figure 22, with refinement concentrated in the neighbourhood of the keyhole slot. The displacement field is approximated with cubic polynomial basis functions. The ground truth constitutive law is the foam elastoplastic model: the elastic softening modulus $E(\kappa)$ of Eq. (20) combined with J2 plasticity and the Voce hardening law of Eq. (23)—the same material used for the training in Sections 1.2.1 and 1.2.2.

3.2.4 Simulating Shear-Coupon Experiments

The specimen simulated in Section 1.2.4 is discretised with a triangular mesh, as shown in Figure 23. Plane stress is modelled with a mixed formulation: the in-plane displacement is approximated with linear (CG1) basis functions, and the out-of-plane normal strain E_{33} is carried as an additional unknown so that the plane-stress condition $S_{33} = 0$ is enforced weakly.

Unlike the previous solid-mechanics examples, the shear coupon develops large rotations within the ligament and is therefore modelled with the finite-strain, total-Lagrangian formulation of Section 3.2 (Eqs. (17)–(18)), with the additive elastic–plastic split $E = E^e + E^p$ of the Green–Lagrange strain. The second Piola–Kirchhoff stress follows the St. Venant–Kirchhoff law on the elastic strain, degraded by the scalar damage,

$$S = g(\bar{D}) \mathbb{C} : (E - E^p), \quad (30)$$

and the weak form pairs the first Piola–Kirchhoff stress $P = FS$ with the gradient of the test function. Plasticity is governed by a J2 (von Mises) yield criterion with associative flow, $f = \sigma_{\text{eq}} - \sigma_y(p) \leq 0$, solved by a radial return on the trial deviatoric stress at each loading step; here p is the accumulated plastic strain (Eq. (25)) and $\sigma_y(p)$ the learned hardening law. The elastic constants are fixed throughout ($E = 115$ GPa from an independent tensile gauge, $\nu = 0.3$).

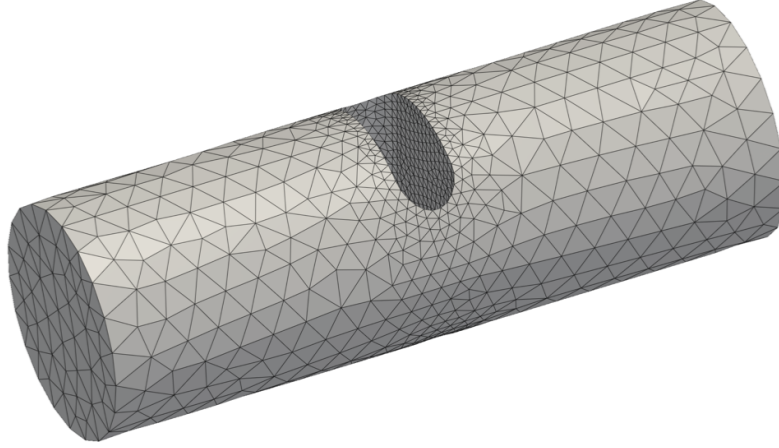


Figure 22. Computational mesh used in the torsion experiment on the slotted cylindrical rod.

The scalar damage $\bar{D} \in [0, 1]$ degrades the stress through $g(\bar{D}) = (1 - \bar{D})^2$. It is driven by the accumulated plastic strain through a local hazard $\Phi_{\text{loc}}(p)$ (the learned damage operator, below). A local damage law of this kind localises into a single element under mesh refinement; we regularise it with an implicit-gradient (nonlocal) filter, smoothing the hazard by a Helmholtz equation with length scale ℓ ,

$$\bar{\Phi} - \ell^2 \nabla^2 \bar{\Phi} = \Phi_{\text{loc}}(p), \quad (31)$$

with natural (Neumann) boundary conditions, and define the nonlocal damage as $\bar{D} = 1 - e^{-\bar{\Phi}}$ (capped at 0.95). The length scale ℓ sets the width of the softening band, making the localisation mesh-objective rather than collapsing onto a single element. Damage is advanced explicitly (staggered): equilibrium at each step is solved with the damage from the previous step, after which $\bar{D}$ is updated from the new plastic state.

ML Model Architecture Two operators are learned, both as monotone neural networks built from the integral of a non-negative activation, so that the relevant physical monotonicity holds by construction; the activation is matched to the qualitative behaviour of each law. The hardening law $\sigma_y(p)$ uses the same integral monotone network as the Brazilian disc test (Section 3.2.2, Eq. (26)): a strictly positive hardening rate obtained from a SoftPlus output is integrated by Gauss–Legendre quadrature, giving a smooth yield stress that rises monotonically from $p = 0$.

The damage hazard $\Phi_{\text{loc}}(p)$ is parameterised by a *rectified integral network*: a single hidden layer of H rectified-quadratic units with non-negative output weights, whose integral over plastic strain gives the cumulative hazard,

$$r(p) = \sum_{j=1}^H s_j \langle p - b_j \rangle_+^2, \quad s_j \geq 0, \quad \Phi_{\text{loc}}(p) = \int_0^p r(s) ds = \sum_{j=1}^H \frac{s_j}{3} \langle p - b_j \rangle_+^3, \quad (32)$$

where $\langle \cdot \rangle_+$ denotes the positive part (rectifier), $\{b_j\}$ are learnable breakpoints and $\{s_j\}$ learnable non-negative slopes (we use $H = 8$, i.e. 16 parameters). This architecture enforces, by construction, the qualitative properties expected of a ductile-damage hazard: $\Phi_{\text{loc}}(0) = 0$; monotone non-decreasing accumulation ($s_j \geq 0 \Rightarrow r \geq 0$); a *hard onset*, since each rectified-quadratic unit is exactly zero below its breakpoint, so $r(p) = 0$ for $p < \min_j b_j$ (a dormant plateau, with no leakage below onset); and a smooth (C^1 -continuous), convex, accelerating growth above the onset. The contrast with the hardening operator is deliberate: hardening integrates a *SoftPlus* activation (a smooth rise from $p = 0$), whereas damage integrates a *rectified-quadratic* activation (a dormant plateau, a hard onset, then smooth accelerating growth), each matched to the physics of the law it represents. Squared rectifiers are used in place of plain ReLUs precisely so that the learned hazard rate is smooth rather than piecewise-linear. Unlike a fixed closed-form rate, the network learns the shape of the hazard directly from the experimental response.

3.3 Transient thermodynamics problem formulation

We consider a transient heat conduction problem on two bodies, where the thermal conductivity of one of the two bodies is a nonlinear function of temperature. This problem is governed by the time-dependent heat equation, expressed as:

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T), \quad (33)$$

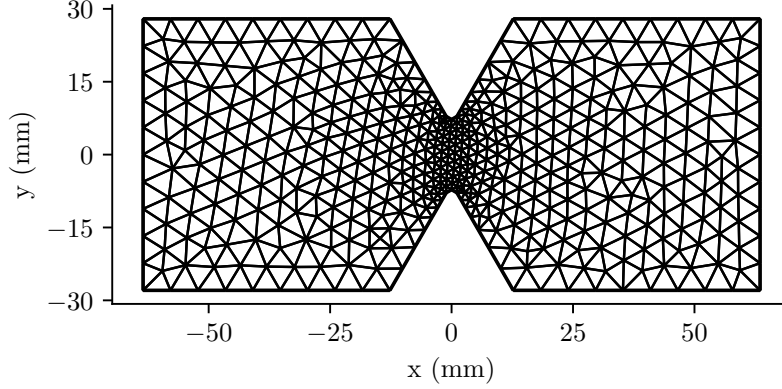


Figure 23. Computational mesh for the shear-coupon experiment.

where T represents the temperature field in Kelvin, ρ denotes the material density measured in kg/m^3 , and c_p is the specific heat capacity, expressed in $\text{J} \cdot \text{kg}^{-1} \cdot \text{K}^{-1}$. The expression $\nabla \cdot (k \nabla T)$ describes the heat flux divergence, while $\frac{\partial T}{\partial t}$ corresponds to the transient variation of temperature. The term k is the thermal conductivity, measured in $\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$, which can be a nonlinear function of temperature and parametrised with a neural network:

$$k = \mathcal{G}_\theta(T). \quad (34)$$

3.3.1 Simulating Transient Heat Conduction Experiments

The two bodies simulated in Section 1.3 are discretised with a tetrahedral mesh shown in Figure 24, where half of the square plate is transparent to show the hole in the circular bottom plate. The temperature field is discretised with linear basis functions.

Ground Truth The ground truth thermal conductivity law is a nonlinear function of the temperature defined as:

$$k = k_r \left(1 + \beta \frac{T - T_r}{T_r} \right)^{-\delta} \quad (35)$$

where T is the temperature; $\beta = 1.0$ is a dimensionless constant; $\delta = 0.62$ is an exponent that characterises the temperature dependence; $k_r = 2.0$ is a reference thermal conductivity; and $T_r = 298.0$ K is the reference temperature. Figure 25 shows the evolution of the temperature field across the domain at four time instants.

ML Model Architecture The ML constitutive model is defined with an MLP with two hidden layers, each containing thirty neurons and using ReLU and Sigmoid as the activation functions. The input of the MLP is the temperature, and the output is the corresponding thermal conductivity, which is then used to solve the PDE.

3.4 Hyper-parameter summary and sensitivity to training epochs

Table 1 summarises the ML hyper-parameters for the four examples in which an operator is trained (the torsion example of Section 1.2.3 reuses pretrained operators with no further training). The displacement-controlled example uses a compact MLP with two hidden layers of thirty neurons each, SiLU hidden activations, and the Adam optimiser with a one-cycle learning-rate schedule. The load-controlled plasticity example uses an integral monotone neural network (Section 3.2.2) with a single hidden layer of thirty-two neurons and a one-cycle schedule.

The shear-coupon example (Section 1.2.4, the two shear-coupon columns of Table 1) is a two-stage, real-data calibration: *Stage 1* learns the hardening law $\sigma_y(p)$ on the rising branch, and *Stage 2* then freezes it and learns the rectified integral network for the damage hazard on the full curve. Both stages minimise the scatter-normalised residual of Eq. (8), with the residuals grouped into branches that are each averaged internally and combined with fixed weights—elastic, transition, and plateau weighted 2 : 1 : 1 in Stage 1, and pre-peak and post-peak weighted 1.5 : 2 in Stage 2—so that branch importance is set explicitly rather than by the number of load steps that fall in each branch. In both stages the gradient is obtained from the firedrake-adjoint, differentiating end-to-end through the finite element solve—including, in Stage 2, the staggered nonlocal-damage update—into the network parameters.

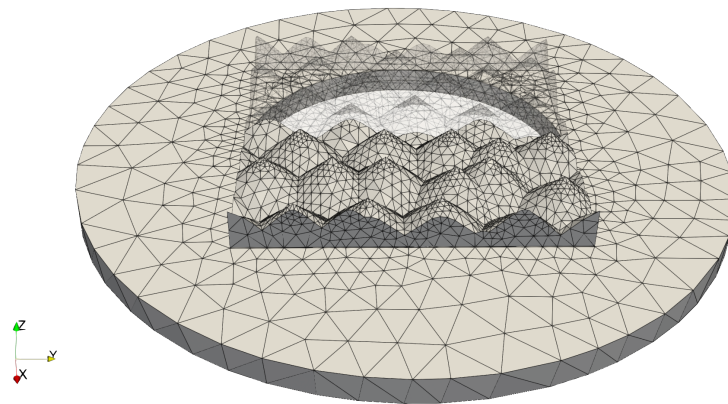


Figure 24. Computational mesh used in the thermal conduction experiment.

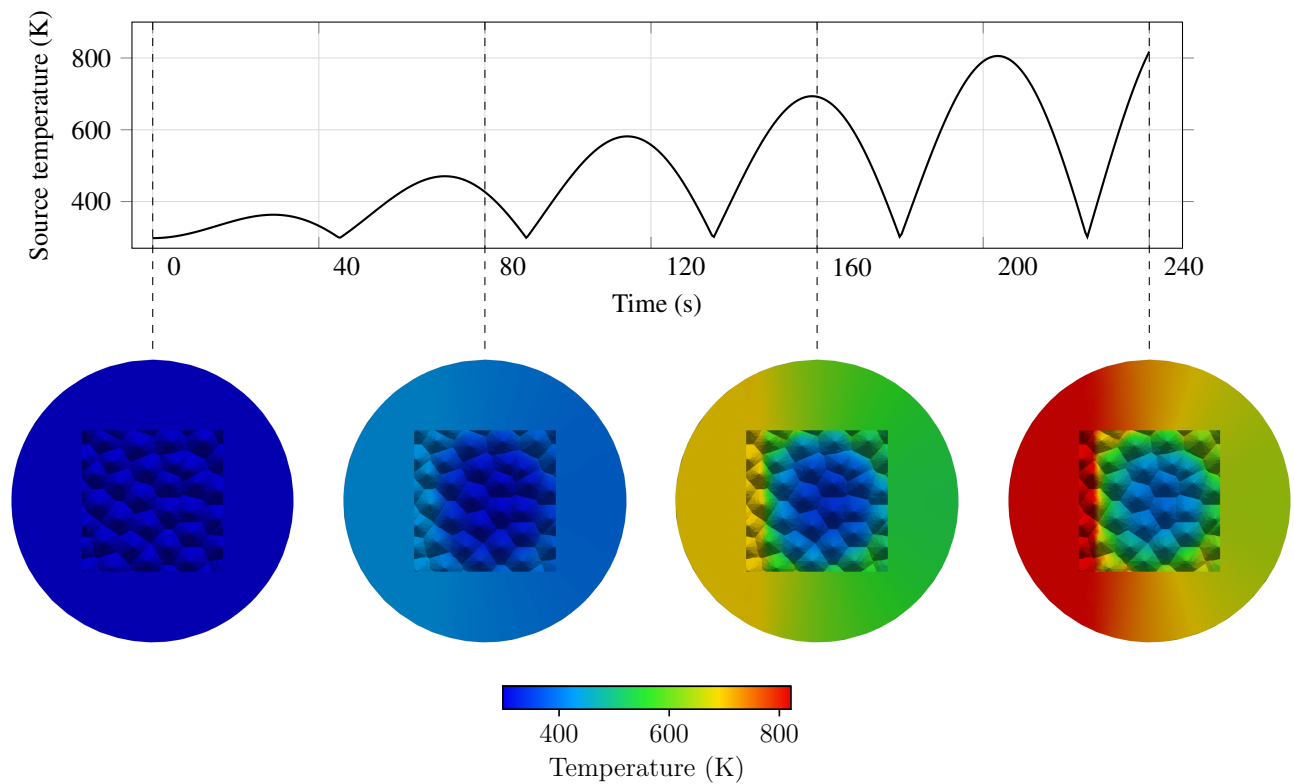


Figure 25. Fluctuating source temperature boundary condition applied at the left boundary (top) and the resulting temperature field on the simulated domain at four time instants (bottom). The temperature progressively diffuses across the two-body domain.

Table 1. Summary of ML hyper-parameters for the four examples in which an operator is trained; the two-stage shear-coupon calibration is shown as its separate hardening (Stage 1) and damage (Stage 2) stages. In the damage stage the two learning rates apply to the network weights (0.005) and to the onset/amplitude scalars (0.02).

Hyper-parameter	Disp.-controlled	Load-controlled	Shear-coupon (hardening)	Shear-coupon (damage)	Thermal
Trainable model(s)	1 MLP	1 monotone NN	1 monotone NN	1 monotone NN	1 MLP
Learned quantity	$E(\kappa)$	$\sigma_y(p)$	$\sigma_y(p)$	$\Phi_{loc}(p)$	$k(T)$
Frozen operator(s)	—	$E(\kappa)$ from Exp. 1	E fixed	E fixed; σ_y	—
Hidden layers	2×30	1×32	1×32	8 units	2×30
Hidden activation	SiLU	SoftPlus	SoftPlus	rect.-quad.	ReLU
Output activation	SoftPlus	SoftPlus (integral)	integral	integral	Sigmoid
Optimiser	Adam	Adam	Adam	Adam	Adam
Learning rate	0.03 (max)	0.05 (max)	0.03	0.005–0.02	0.025
Epochs	200	400	60	60	100
Training samples	6 load steps	15 load steps	23 steps	80 steps	12 time steps
Noise level	1% (scalar forces)	1% (nodal disp.)	real data (4 spec.)	—	2% (nodal temp.)
LR scheduler	One-cycle	One-cycle	—	—	—

The number of training epochs is the most impactful discrete hyper-parameter, and its adequacy is established directly from the training histories reported for each example. In the displacement-controlled (Figure 4), load-controlled (Figure 7), and transient heat conduction (Figure 17) examples, the training and test losses decrease monotonically and then flatten into a plateau over the final epochs. Throughout training the test loss tracks the training loss closely, indicating generalisation rather than overfitting, and the late-epoch plateau shows that the solution has effectively converged well before the final epoch. The reported epoch counts therefore lie within this converged regime, so the learned operators are insensitive to the precise stopping point.

Acknowledgements

N.B. was supported by the Eric and Wendy Schmidt AI in Science Postdoctoral Fellowship, a Schmidt Futures program. D.A.H. was supported by the Engineering and Physical Sciences Research Council (EPSRC) under grants EP/W029731/1 and EP/W026066/1, and by the Science and Technology Facilities Council (STFC, UKRI) under grant UKRI/ST/B000495/1.

Data availability

The experimental data for the real shear-coupon example are from the Second Sandia Fracture Challenge²⁵ and are included in the code repository below. All other data analysed in this study are synthetic and are generated by the accompanying code (see Code availability).

Code availability

The numerical examples were run with the open-source finite element framework Firedrake¹⁵. Instructions to run the examples in this paper, together with the Python scripts, meshes, and data required to reproduce all results, are archived on Zenodo ([10.5281/zenodo.21251671](https://doi.org/10.5281/zenodo.21251671)).

Author contributions

A.F.; N.B.: Conceptualization; Methodology; Formal analysis; Software; Validation; Writing - original draft; Writing - review & editing. D.A.H.: Writing - review & editing.

Competing interests

The authors declare no competing interests.

References

1. Karniadakis, G. E. *et al.* Physics-informed machine learning. *Nat. Rev. Phys.* **3**, 422–440 (2021).
2. Bouziani, N., Ham, D. A. & Farsi, A. Differentiable programming across the PDE and Machine Learning barrier. *arXiv preprint arXiv:2409.06085* (2024).

3. Quarteroni, A., Gervasio, P. & Regazzoni, F. Combining physics-based and data-driven models: advancing the frontiers of research with scientific machine learning. *Math. Model. Methods Appl. Sci.* **35**, 905–1071, DOI: [10.1142/S0218202525500125](https://doi.org/10.1142/S0218202525500125) (2025). <https://doi.org/10.1142/S0218202525500125>.
4. Belbute-Peres, F. d. A., Economon, T. D. & Kolter, J. Z. Combining Differentiable PDE Solvers and Graph Neural Networks for Fluid Flow Prediction, DOI: [10.48550/arXiv.2007.04439](https://arxiv.org/abs/2007.04439) (2020). ArXiv:2007.04439 [physics, stat].
5. Bouziani, N. & Ham, D. A. Physics-driven machine learning models coupling PyTorch and Firedrake. In *ICLR Workshop on Physics for Machine Learning*, DOI: [10.48550/arXiv.2303.06871](https://arxiv.org/abs/2303.06871) (2023).
6. Crilly, A., Duhig, B. & Bouziani, N. Learning closure relations using differentiable programming: An example in radiation transport. *J. Quant. Spectrosc. Radiat. Transf.* 108941 (2024).
7. Brunton, S. L., Proctor, J. L. & Kutz, J. N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proc. Natl. Acad. Sci.* **113**, 3932–3937, DOI: [10.1073/pnas.1517384113](https://doi.org/10.1073/pnas.1517384113) (2016).
8. Rao, C. *et al.* Encoding physics to learn reaction–diffusion processes. *Nat. Mach. Intell.* **5**, 765–779, DOI: [10.1038/s42256-023-00685-7](https://doi.org/10.1038/s42256-023-00685-7) (2023).
9. Haghighat, E., Abouali, S. & Vaziri, R. Constitutive model characterization and discovery using physics-informed deep learning. *Eng. Appl. Artif. Intell.* **120**, 105828, DOI: [10.1016/j.engappai.2023.105828](https://doi.org/10.1016/j.engappai.2023.105828) (2023).
10. Li, Z. *et al.* Fourier Neural Operator for Parametric Partial Differential Equations. In *International Conference on Learning Representations* (2021).
11. Li, Z. *et al.* Physics-Informed Neural Operator for Learning Partial Differential Equations, DOI: [10.48550/arXiv.2111.03794](https://arxiv.org/abs/2111.03794) (2023). ArXiv:2111.03794 [cs, math].
12. Bartolucci, F. *et al.* Representation Equivalent Neural Operators: a Framework for Alias-free Operator Learning. In *Thirty-seventh Conference on Neural Information Processing Systems* (2023).
13. Raissi, M., Perdikaris, P. & Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707, DOI: [10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045) (2019).
14. Bouziani, N. & Boullé, N. Structure-preserving operator learning. *arXiv preprint arXiv:2410.01065* (2024).
15. Ham, D. A. *et al.* *Firedrake User Manual*. Imperial College London and University of Oxford and Baylor University and University of Washington, first edition edn., DOI: [10.25561/104839](https://doi.org/10.25561/104839) (2023).
16. Bouziani, N. & Ham, D. A. Escaping the abstraction: a foreign function interface for the Unified Form Language [UFL]. In *NeurIPS Workshop on Differentiable Programming*, DOI: [10.48550/arXiv.2111.00945](https://arxiv.org/abs/2111.00945) (2021).
17. Paszke, A. *et al.* PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, vol. 32 (Curran Associates, Inc., 2019).
18. Bradbury, J. *et al.* JAX: composable transformations of Python+NumPy programs. <http://github.com/google/jax> (2018). Version 0.3.13.
19. Ataei, M. & Salehipour, H. XLB: A differentiable massively parallel lattice Boltzmann library in Python. *Comput. Phys. Commun.* **300**, 109187, DOI: [10.1016/j.cpc.2024.109187](https://doi.org/10.1016/j.cpc.2024.109187) (2024). ArXiv:2311.16080 [physics].
20. Holl, P., Koltun, V. & Thuerey, N. Learning to Control PDEs with Differentiable Physics, DOI: [10.48550/arXiv.2001.07457](https://arxiv.org/abs/2001.07457) (2020). ArXiv:2001.07457 [physics, stat].
21. Joglekar, A. S. & Thomas, A. G. R. Machine learning of hidden variables in multiscale fluid simulation. *Mach. Learn. Sci. Technol.* **4**, 035049, DOI: [10.1088/2632-2153/acf81a](https://doi.org/10.1088/2632-2153/acf81a) (2023). Publisher: IOP Publishing.
22. Rackauckas, C. *et al.* Universal Differential Equations for Scientific Machine Learning, DOI: [10.48550/arXiv.2001.04385](https://arxiv.org/abs/2001.04385) (2021). ArXiv:2001.04385.
23. Brenner, S. C. & Scott, L. R. *The Mathematical Theory of Finite Element Methods*, vol. 15 of *Texts in Applied Mathematics* (Springer New York, New York, NY, 2008), 3 edn.
24. Farsi, A. *et al.* Full deflection profile calculation and young’s modulus optimisation for engineered high performance materials. *Sci. Reports* **7**, 46190, DOI: [10.1038/srep46190](https://doi.org/10.1038/srep46190) (2017).
25. Boyce, B. L. *et al.* The second Sandia Fracture Challenge: predictions of ductile failure under quasi-static and moderate-rate dynamic loading. *Int. J. Fract.* **198**, 5–100, DOI: [10.1007/s10704-016-0089-7](https://doi.org/10.1007/s10704-016-0089-7) (2016).

26. Schmidt, M. & Lipson, H. Distilling free-form natural laws from experimental data. *Science* **324**, 81–85, DOI: [10.1126/science.1165893](https://doi.org/10.1126/science.1165893) (2009).
27. Cranmer, M. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582* DOI: [10.48550/arXiv.2305.01582](https://doi.org/10.48550/arXiv.2305.01582) (2023).
28. Kronberger, G., de França, F. O., Burlacu, B., Haider, C. & Kommenda, M. Shape-constrained symbolic regression—improving extrapolation with prior knowledge. *Evol. Comput.* **30**, 75–98, DOI: [10.1162/evco_a_00294](https://doi.org/10.1162/evco_a_00294) (2022).
29. Alnæs, M. S., Logg, A., Ølgaard, K. B., Rognes, M. E. & Wells, G. N. Unified form language: A domain-specific language for weak formulations of partial differential equations. *ACM Trans. Math. Softw.* **40**, DOI: [10.1145/2566630](https://doi.org/10.1145/2566630) (2014).
30. Hinze, M., Pinnau, R., Ulbrich, M. & Ulbrich, S. *Optimization with PDE Constraints*, vol. 23 of *Mathematical Modelling: Theory and Applications* (Springer Netherlands, Dordrecht, 2009). ISSN: 1386-2960.
31. Mitusch, S., Funke, S. & Dokken, J. dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake. *J. Open Source Softw.* **4**, 1292, DOI: [10.21105/joss.01292](https://doi.org/10.21105/joss.01292) (2019).